DevOps Interview — Complete Knowledge Base

⚡ ⚡ Architect Compendium

Q: What is the fundamental difference between High Availability and Disaster Recovery?

The most mis-answered foundational question

Dimension	High Availability (HA)	Disaster Recovery (DR)
Scope	Within a single Region (cross-AZ)	Across different geographic Regions
Primary Goal	Minimize downtime during AZ failure	Restore service after Region-wide event
AWS Examples	Multi-AZ EC2, ALB	Route 53 Health Checks, S3 CRR

- ❌ Bad: "HA means the app stays up; DR is when it falls back to another region."
- ✓ Delightful: HA is scoped to **AZs within one region**. DR is scoped to **different geographic regions**. Confusing them signals you haven't operated at enterprise scale.
- **So What?** A total AZ failure = HA handles it, no human intervention. A total region failure = DR, a deliberate architectural strategy.

Q: Walk me through a production-grade 3-tier Highly Available architecture.

Layer-by-layer HA walkthrough — the "delightful" answer format

- **External ALB:** Routes across AZs. Inherently HA — no SPOF.
- **Web/App Tier (EC2 in ASG):** Single EC2 = SPOF. Deploy across ≥2 AZs in ASG. AZ fails → traffic re-routes automatically.
- **Internal ALB:** Decouples web and app tiers with identical HA properties.
- **Data Tier (DynamoDB):** Regional service, auto-replicates across 3 AZs. AZ failure is transparent.
- **So What?** Complete data-center failure = *non-event* for users. Zero manual intervention.

Q: Define RTO and RPO. Compare all 4 DR strategies.

The "trap question" — RPO is always measured in TIME, not data volume

- **RTO:** Max acceptable time offline. Measured in minutes/hours/days.
- **RPO:** Max acceptable data loss as a point in *time*. Common mistake: measuring in megabytes. Wrong — it's always time.

Strategy	RTO / RPO	How It Works
Backup & Restore	Hours / Days	S3 backups restored to new region. Highest RTO, lowest cost.
Pilot Light	Min / Hours	Core DB live + replicated; app servers "off" until trigger.
Warm Standby	Min / Min	Scaled-down full env always running. Scale up on activation.
Active-Active	Real-time	Full traffic in 2+ regions. Zero RTO/RPO, highest cost.

Q: Contrast Monolithic vs. Microservices. Explain Event-Driven Architecture.

Show fault isolation and EDA asynchrony — not just definitions

- **Monolith:** Single coupled unit. One fail → redeploy everything. No independent scaling.

- **Microservices:** Independent services, independently deployed/scaled, polyglot.
- **Real Flow — store.com (ALB Path-Based):**
  - `/browse` → EC2+DynamoDB (Python)
  - `/purchase` → EKS+Aurora (Go) — isolated for integrity
  - `/return` → Lambda (serverless, spiky)
  - If /browse crashes, /purchase + revenue unaffected. *Fault isolation.*
- ❌ Bad EDA: "API GW → Lambda → DynamoDB." That's *synchronous*.
- ✓ Delightful EDA: API GW → **SQS** → Lambda → DynamoDB. SQS decouples. Independent scaling, built-in retry, no idle waste.

Trait	Synchronous	EDA (Async)
Scaling	All layers must match	Components scale independently
Retries	Hard-coded in client	Built-in queue retry
Cost	Over-provision for spikes	Pay only for events processed

Q: Compare SQL vs. NoSQL from a Solutions Architect perspective.

Distinguish by schema approach and consistency model

Property	SQL (Relational)	NoSQL
Schema	Schema-on-write — rigid	Schema-on-read — flexible
Consistency	ACID transactions	Eventual (configurable)
Scaling	Vertical primary	Horizontal by design
Best For	Complex joins, financial txns	High-throughput, low-latency, variable
AWS	Aurora, RDS	DynamoDB

- **Architect's Rule:** Never say "NoSQL is better." Choose based on access patterns. ACID joins → SQL. Single-digit ms at millions RPS → DynamoDB.

Q: Strategic GenAI use cases. Explain MCP. Differentiate A2A, MCP, and RAG.

Categorize GenAI use cases — "chatbots" alone is a junior answer

- **Customer Experience:** Agentic assistants that take action (not just answer)
- **Productivity:** Content creation, report gen, search summarization — measurable ROI
- **Cutting Edge (MCP):** Automated cost analysis, code-quality evaluation against architectural best practices
- **Model Context Protocol (MCP):** Standardized JSON-RPC protocol for LLMs to interact with tools without custom-coding each integration.
  - **Pre-MCP:** Manually define every API URL, header, payload per tool
  - **MCP:** Server exposes schemas. Client discovers dynamically. LLM chooses tool based on purpose.
  - **Key Innovation:** Dynamic Discovery at runtime

Protocol	What It Does
A2A	Agent-to-Agent communication (orchestration between agents)

MCP	Standardized protocol connecting agents/LLMs to tools and data
RAG	Retrieval-Augmented Generation — grounding LLM in external data without retraining

---

Q: Continuous Delivery vs. Deployment. Senior-level pipeline. CI/CD vs. GitOps.

Know the human-button distinction and the push vs. pull model

- **Delivery:** Always deployable; *manual trigger* to prod.
- **Deployment:** Every passing build auto-deploys. Zero human intervention.
- **Senior Pipeline (Source→Build→Test→Deploy):**
  - **IaC:** Non-negotiable — Terraform for consistency
  - **Automated Testing:** CodeBuild catches errors pre-staging
  - **DevSecOps:** Shift-left — secret scanning + vuln assessment as pipeline gates
  - **Multi-Account:** Dev→Test→Prod isolation minimizes blast radius

Model	Traditional CI/CD	GitOps
Direction	Push — tool pushes to env	Pull — controller pulls from Git
Truth Source	CI/CD tool state	Git repo (declarative)
Controller	Jenkins, GH Actions, Harness	ArgoCD, Flux
Drift	Manual/scheduled	Continuous auto-reconciliation

Q: Why EKS? HPA vs. Cluster Autoscaler vs. Karpenter. DaemonSets and Sidecars.

Three layers of scaling, and when each kicks in

- **Why EKS:** AWS manages control plane (API server, etcd) across AZs. Managing this yourself is the hardest part of K8s ops — EKS eliminates it.

Scaler	What It Scales	Trigger	Notes
HPA	Pods	CPU/Memory/custom	Doesn't add nodes
Cluster AS	Nodes (via ASG)	Pending pods	Works with EC2 ASGs
Karpenter	Nodes (directly)	Pending pods	Bypasses ASGs; right-sized instantly. Faster + cheaper.

- **DaemonSet:** One pod per node. Cluster-wide agents — Fluent Bit, node-exporter.
- **Sidecar:** Same pod as app. Pod-specific — Envoy proxy, log offloading.
- **Monitoring stack:** Prometheus+Grafana (metrics), Fluent Bit/CloudWatch Insights/ELK (logs). Not naming yours is a red flag.

Q: Lambda cold starts, scaling, security. SQS vs. SNS vs. EventBridge.

CPU is indirect, cold starts are solvable, messaging services are distinct

- **Cold Start:** Latency on new execution env. Mitigate: **Provisioned Concurrency** or Custom Container Images.
- **CPU:** Can't set directly. Increase **memory** → AWS scales CPU proportionally. More memory = faster = lower cost.

- **Lambda vs Fargate:** Lambda = instant burst. Fargate = slower but handles >15 min tasks.
- **Security:** Least-privilege IAM per function, KMS for env vars, VPC integration.

Service	Pattern	Use Case
SQS	Queue (point-to-point)	1:1 decoupling, backpressure, retry
SNS	Pub/sub (fan-out)	1:many broadcast notifications
EventBridge	Event bus (content-based)	Filter/route from AWS, SaaS, custom apps

---

Q: Interview strategy: The 'Question Behind the Question' and critical mistakes.

The "So What?" layer separates architects from engineers

- **Trait 1 — Question Behind the Question:** When asked "What is RPO?", the real test is whether you know it's measured in *time*, not data volume. Pivot to the underlying principle.
- **Trait 2 — System Design Pivot:** Don't say "SQS is a queue." Explain how you used SQS to decouple an ordering system and absorb a 10x Black Friday spike. That's the delight factor.
- **Trait 3 — Big Picture:** Show how services interact to solve a business problem. Walk through flows, not definitions.
- ❌ Rambling: 5 min on a VPC question = can't communicate with executives. Use **3-2-1**: 3 points, 2 minutes, 1 summary.
- ❌ Generic: "I would use best practices" is not an answer. Name service, trade-off, consequence.
- ❌ Jumping to solutions: Ask requirements first — RTO, RPO, budget, scale, compliance. Skipping = junior.
- ❌ "The best" service: A Principal Architect selects based on requirements — cost, latency, scale — never preference.

⭐ STAR Interview Deep Dives

Q: How can you make your application scalable for a big traffic day?

Framework: Plan ahead, scale at every layer

- **Pre-warm ALBs** via AWS Support request for known traffic spikes
- **Auto Scaling Groups:** min/desired >1; use Warm Pools for pre-initialized instances
- **Scheduled Scaling** when traffic time is predictable
- **Lightweight AMIs** reduce provisioning time during burst scale-out
- **RDS Proxy** for connection pooling — prevents DB overload
- **AWS Countdown:** Simulate with your account team
- **Limits:** Check and raise soft limits; consider multi-region for extreme scale

⭐ **STAR — Enterprise Pharma Mobile CI/CD**

S: Mobile CI/CD burst load during global release windows — multiple teams triggering iOS/Android builds during quarterly pharma compliance deadlines.

T: Ensure self-hosted macOS runner fleet + AWS infra didn't bottleneck on high-traffic release days while meeting GxP audit requirements.

A: Dynamic runner scaling via GitHub Actions runner groups with concurrency limits + queue-aware autoscaling. Fastlane parallel lanes + S3 dependency caching. K8s HPA on custom queue-depth metrics. Scheduled pre-warming policies.

R: iOS build: 45→15 min. Release cycle: 2 days→15 min. Developer NPS: +45. Zero degradation on peak days.

Q: What is an AI agent?

Definition & Key Properties

- **Definition:** Software that autonomously chooses the best sequence of actions to complete a goal — distinct from conditional workflows or standalone LLMs.
- **Reasoning Engine:** Uses LLM to decide which tools to invoke and in what order
- **Feedback Loop:** Perceives context → acts → observes results → iterates
- **AWS:** Lambda/EC2 for execution, Bedrock for LLM, MCP for tool integration

⭐ **STAR — AutoDocs (SKCloudOps)**
S: Contributors confused AutoDocs with a simple API wrapper — needed clear architectural distinction.
T: Define what makes it an agent and design agentic behavior as opt-in.
A: Dual-engine: heuristic default (no API key) + AI opt-in activating full agent loop (perception→reasoning→action→feedback). Agent reads git diff, LLM decides which files need docs, generates, evaluates quality, iterates.
R: 91% statement coverage, 86 tests. Dual-engine pattern became reference for AI CI/CD Guardian. Key differentiator on GitHub Marketplace.

Q: How do you achieve disaster recovery for your cloud application? (STAR)

Framework: RTO/RPO drives strategy selection

Strategy	RTO	RPO	Best For
Backup & Restore	Hours	Hours	Non-critical / cost-sensitive
Pilot Light	Min–Hours	Minutes	Core services always on
Warm Standby	Minutes	Seconds	Business-critical apps
Active-Active	Near zero	Near zero	Enterprise critical / pharma

⭐ **STAR — Pharma DevSecOps Pipeline DR**
S: GxP compliance required CI/CD platform (Vault clusters, GitHub OIDC, artifacts) to meet strict RTO/RPO. Pipeline outage during regulated release = audit findings.
T: Design DR without static credentials or manual runbooks.
A: Warm Standby: Vault replicated across 2 regions with KMS auto-unseal. GitHub OIDC = no long-lived secrets. Terraform state in S3 with versioning + CRR. Recovery = `terraform apply`, not a runbook.
R: Automated + auditable recovery. Zero static-credential exposure during failover. Architecture became DR standard in pharma compliance docs.

Q: How do you secure your application on the cloud? (STAR)

Framework: Defense in Depth — every layer

- **Edge:** AWS WAF — SQL injection, XSS, DDoS
- **Auth:** Cognito for users; OIDC/JWT for machine identity
- **Transit:** SSL/TLS everywhere; ACM certificates; API Gateway enforces HTTPS
- **Compute:** IAM least privilege; Inspector for Lambda scanning
- **Secrets:** Secrets Manager or Vault — never env vars
- **Data at rest:** KMS for RDS, S3, EBS
- **Audit:** CloudTrail, GuardDuty, Security Hub

⭐ **STAR — Zero-Static-Credentials Architecture**
S: Both client (pharma, GxP) and client (financial) required zero-tolerance security. Legacy used static AWS keys in pipelines.
T: Eliminate all static credentials while maintaining velocity + compliance.
A: Vault + GitHub OIDC — every pipeline authenticates via short-lived OIDC tokens. IAM least-privilege via Terraform policy-as-code. Built AI CI/CD Guardian with 42 heuristic patterns across 16 build systems to detect credential leaks.

R: Passed pharma audit with zero static-credential findings. Pattern became reusable GH Actions template. Onboarding: 2 weeks→2 hours.

Q: Describe an architecture you designed. (STAR)

Interviewer goal: business problem understanding + deep-dive readiness

- Pick a **real system** — be prepared for: How does it scale? How secured? What would you change?
- Interviewer will **probe the edges** with follow-ups

⭐ **STAR — Enterprise Pharma Mobile CI/CD Platform**

S: Mobile teams releasing iOS manually — Xcode on laptops, 2-day cycles, no audit trail, no GxP compliance.
T: End-to-end mobile CI/CD platform meeting pharma compliance, eliminating manual steps, scaling globally.
A: Self-hosted macOS runner fleet on GitHub Actions + Fastlane for build/test/sign/deploy. Vault+OIDC for secrets — Apple certs fetched at runtime. Artifacts in S3. Branch protection, multi-party approvals, Splunk logs for GxP audit. RDS Proxy for metadata service.
R: Release: 2 days→15 min. Build: 45→15 min. Onboarding: 2 weeks→2 hours. NPS: +45. Became standard platform at client.

Q: Biggest challenge faced during designing your application on the cloud. (STAR)

Two strong angles: Scaling or Cost

- **Scaling:** When standard ASG can't react fast enough — Warm Pools + pre-warming
- **Cost:** Unexpected bills — CloudWatch Insights, Compute Optimizer, Spot, Cost Explorer

⭐ **STAR — Vault + OIDC Bootstrap Problem**

S: Ephemeral GitHub Actions runners + Vault = chicken-and-egg: how authenticate without a pre-placed credential? Static creds off the table for compliance.
T: Solve runner auth bootstrap with zero static secrets — auditable, least-privilege, reusable.
A: Three-hop trust chain: GH Actions OIDC JWT → AWS STS AssumeRole (no static keys) → Vault AWS auth backend → short-lived Vault token. IAM trust scoped to repos/branches/envs. Every hop logged (GH audit, CloudTrail, Vault audit).
R: Zero static credentials. Three-hop chain became reusable composite action, adopted by 6 teams in 3 months. Cited as resolved finding in GxP audit.

Q: How do you pick one service versus another? (STAR)

Framework: Requirements-first, then trade-off analysis

- Always **ask for requirements** first: traffic volume, latency SLA, security, cost, uptime

Decision	Choose A when...	Choose B when...
API GW vs ALB	External API, auth, rate limiting	Internal microservice, WebSocket, gRPC
Serverless vs EC2	Burst, short-lived tasks	Long-running, stateful, GPU
EventBridge vs SQS/SNS	Cross-service/account routing	Point-to-point queue, fan-out pub/sub
EKS vs ECS vs Fargate	K8s portability, complex workloads	Simpler ops; Fargate = no nodes

⭐ **STAR — Transit Gateway vs VPC Peering**

S: Connect multiple VPCs (shared services, dev, staging, prod) with centralized security inspection for pharma compliance.
T: Recommend multi-VPC model that scales, supports audit, avoids operational overhead.
A: Evaluated across 4 axes: complexity (N^2 vs hub), routing (non-transitive vs centralized), cost, compliance (TGW + Network Firewall = single inspection point).

R: Standardized on TGW + shared services VPC. Security team got single firewall inspection point. Cross-account complexity → centralized route table updates.

Q: What is your favorite AWS service? How will you improve it?

Answer: IAM OIDC Federation / Roles Anywhere

- **Why:** Represents a philosophy — workload identity from verifiable context, not a possessed secret. This single capability unlocked the entire zero-static-credentials architecture across Vault, GitHub Actions, and AWS.
- **Improvement 1:** Richer claim mapping — custom OIDC claims → IAM condition keys directly, no Lambda authorizers
- **Improvement 2:** Claim-level CloudTrail attribution — surface originating OIDC claims for complete audit chains
- **Improvement 3:** Cross-provider claim normalization — standardized schema for GitHub/GitLab/Buildkite so platform teams write portable IAM policies

---

Q: What is AWS Service X? (Handling unknown services)

Framework: Definition → Properties → Composition

- **Example — Lambda:** "Serverless compute that runs code without provisioning servers. Scales automatically, HA, pay per compute time."
- **Composition:** API Gateway (HTTP), SQS (queue), EventBridge (events), S3 (object events), Vault via OIDC
- **When you don't know:** Be transparent. Reason from first principles — problem space, pricing model, how it composes with IAM/CloudWatch/VPC.

Q: AWS Services Cheat Sheet — Services to know cold

Structured definition pattern for each service

Service	One-line Definition
API Gateway	Managed REST/HTTP/WebSocket API creation, publishing, and security at scale
ALB	Layer 7 LB routing HTTP/S by path, host, or headers to targets
Transit Gateway	Hub-and-spoke transit hub connecting VPCs + on-prem at scale
EKS	Managed K8s control plane — no etcd or CP node management
Bedrock	Managed GenAI service for building apps with foundation models via API
EventBridge	Serverless event bus routing events between AWS, SaaS, and custom apps
RDS Proxy	Managed DB proxy pooling connections to reduce RDS/Aurora load
Secrets Manager	Managed secrets storage with automatic rotation
Inspector	Automated vulnerability scanning for EC2, Lambda, container images
GuardDuty	ML-based threat detection analyzing CloudTrail, Flow Logs, DNS

▶ Video Quick-Reference

Q: Scalability for big traffic day (Quick Reference)

- **Load Balancers:** Pre-warm via AWS Support request for sudden spikes
- **ASG:** Min/desired >1. Scheduled scaling for known times. Warm Pool of pre-initialized instances.
- **Optimization:** Lightweight AMIs for faster provisioning. RDS Proxy for connection pooling.
- **Preparation:** AWS Countdown with account team. Check/raise soft limits. Multi-region for extreme traffic.

Q: AI agent definition (Quick Reference)

- Autonomously chooses best actions to complete a task — distinct from conditional workflows or standalone LLMs.
- Uses LLM to determine which tools, in what order, how many times to interact.
- Implementation: Lambda/EC2, Bedrock for LLM, MCP for tool integration.

Q: Disaster recovery strategies (Quick Reference)

- Strategy depends on RTO and RPO.
- Backup & Restore (highest RTO/RPO) → Pilot Light → Warm Standby → Multi-site Active-Active (lowest, ideal for enterprise despite higher cost).

Q: Cloud security (Quick Reference)

- Defense in depth at every layer.
- Cognito for auth + SSL/TLS via ACM + API Gateway.
- WAF for edge protection (SQLi, XSS).
- Inspector for Lambda scanning + IAM least privilege.
- Secrets Manager + KMS for data at rest.

Q: Describe a system YOU designed (Quick Reference)

- Choose a project you actually implemented.
- Interviewer wants: business problem understanding, how you worked backwards, ability to answer deep-dive follow-ups on scalability and security.

Q: Biggest cloud design challenge (Quick Reference)

- **Scaling:** High-burst traffic beyond standard ASG — implement warm pools + pre-warming.
- **Cost:** Unexpected AWS bills — CloudWatch Insights, Compute Optimizer, Spot, Cost Explorer.

Q: Picking one service over another (Quick Reference)

- Ask for requirements: traffic volume, latency, security, cost, uptime.
- Be ready to contrast: API GW vs ALB, Serverless vs EC2, EventBridge vs SQS/SNS, EKS vs ECS vs Fargate.

Q: Favorite AWS service + improvements (Quick Reference)

- Pick a service you've used extensively.
- Research the public AWS Roadmap on GitHub — pick a highly-voted feature request to discuss.

Q: Handling unknown AWS services (Quick Reference)

- Start with official definition, then key properties.
- Example (Lambda): "Serverless compute, no server management, auto-scales, HA, pay-per-use."
- When you don't know: be transparent, reason from first principles.

👤 Introduction & Career Strategy

Q: How are you / How are you doing?

"I'm doing well, thank you. Excited to be here and looking forward to our conversation."

Q: Would you like to introduce yourself first?

"I'm a Senior DevOps/Cloud Engineer with 15+ years of experience. Currently embedded at client Pharmaceuticals as part of my employerant, managing AWS infrastructure, Kubernetes clusters, and CI/CD pipelines at scale. I've led platform engineering initiatives that reduced service onboarding from 2 weeks to 2 hours, and I'm now targeting Cloud/Platform Architect roles where I can drive broader infrastructure strategy."

Q: What's AI vs purely you?

Architecture decisions, cross-team communication, security review, incident response, stakeholder conversations = human judgment. AI accelerates execution; I own thinking + accountability.

Q: Aspirations in next 5 years?

Move into **Platform/Cloud Architect** in 2-3 years. Own platform strategy — golden paths, cloud standards, org-wide infra decisions. In 5 years: leading platform engineering at Director/Principal level.

Q: How to reduce friction between DevOps and developers?

- Self-service **internal developer platforms**
- Golden path templates (TF modules, Helm charts, pipeline templates)
- Clear documentation + runbooks
- Embed with dev teams periodically
- Shift-left (IaC, scanning, observability early)
- Feedback loops — developer NPS, retros, office hours

Q: Goals after 5-10 years? How do you see DevOps industry?

Principal Architect or Engineering Director. DevOps evolves into **Platform Engineering** + Developer Experience. AI automates repetitive parts; need for deep systems understanding + architectural judgment grows.

Q: Advice for freshers wanting DevOps career?

- Start with Linux + networking
- Learn one cloud deeply (AWS)
- Hands-on Kubernetes
- Terraform from day one
- Build in public — GitHub, open source, blog
- Understand *why* tools exist
- First cert (CKA, AWS SAA) + real projects
- Find a community

Q: Anything you'd like to ask?

"Could you share the biggest platform engineering challenges your team is currently working through? I'd love to understand where I could have the most impact."

☁️ AWS Architecture & Core Compute

Q: How would you architect a fully available and fault-tolerant application on AWS?

- **Compute**: EKS multi-AZ node groups or ASG with EC2
- **Networking**: VPC with public/private subnets across 2+ AZs; ALB for distribution
- **Database**: RDS Multi-AZ or Aurora with read replicas
- **Storage**: S3 with versioning and cross-region replication
- **DNS**: Route 53 with health checks and failover routing
- **Caching**: ElastiCache
- **Observability**: CloudWatch, Grafana, SNS alerting
- **Security**: IAM least-privilege, SGs, NACLs, Secrets Manager, WAF

Q: Are you sure about that?

Yes. IGW is VPC-level. Public vs private is determined entirely by the route table — whether 0.0.0.0/0 points to IGW.

Q: Have you worked with hybrid networking in AWS?

Yes — Site-to-Site VPN and Direct Connect at client's environment.

Q: Why do you think it is so?

VPC Peering has no routing hub concept — each peer is a direct 1:1 relationship with no transitivity. TGW was purpose-built to solve mesh complexity at enterprise scale.

Q: Have you worked with disaster recovery?

Yes — cross-region DR strategies for critical workloads at client on AWS.

Q: What DR solution would you design?

- **Backup & Restore**: Cheapest, highest RTO. S3 snapshots, RDS backups.
- **Pilot Light**: Core services (DB replication) always on; compute scales on failover.
- **Warm Standby**: Scaled-down prod in DR; faster failover.
- **Active-Active**: Both regions serve live traffic; lowest RTO/RPO, highest cost.

Q: How to design for both regions active at once with separate access control?

- Route 53 latency/weighted routing to split traffic
- IAM roles + SCPs for region-specific access
- DynamoDB Global Tables or Aurora Global Database
- Cross-region KMS replication
- Separate accounts per region with cross-account roles
- CI/CD deploying to both regions simultaneously

Q: Have you worked with the Well-Architected Framework?

Yes — reviewed workloads at client to identify gaps in reliability and cost optimization.

Q: List all pillars and summarize each?

- **Operational Excellence**: IaC, observability, runbooks, post-mortems
- **Security**: IAM, encryption, threat detection, incident response
- **Reliability**: Multi-AZ, auto-scaling, backups, chaos engineering
- **Performance Efficiency**: Right-sizing, serverless, caching, benchmarking
- **Cost Optimization**: RIs, Spot, rightsizing, lifecycle policies
- **Sustainability** (6th): Minimize environmental impact, reduce idle, efficient instances

---

Q: Challenges with open source?

- No built-in remote run — use GitHub Actions
- Basic workspace management
- No built-in policy (OPA/Sentinel) — use Checkov
- State locking via S3+DynamoDB works but needs setup

Q: Will you still go through all iterations?

Yes — plan is non-negotiable. For true emergencies, break-glass process with direct apply + mandatory post-incident review.

Q: How ensure devs don't overwrite each other's work?

DynamoDB state locking. Concurrent apply fails with lock error. Workspaces isolate per env. PRs require plan review.

Q: Info on enterprise workspaces?

TFC/Enterprise workspaces: separate variables, RBAC, run history, VCS triggers, fully isolated state, audit logs. Much more powerful than OSS workspaces.

Q: How much of your work is driven by AI?

~40-50% AI-assisted — code generation, boilerplate, docs, troubleshooting research, PR descriptions, test cases.

Q: Do you use Cursor or Claude?

Claude for code review, Terraform modules, documentation, interview prep. GitHub Copilot for IDE coding.

Q: Fetch all EC2 across all accounts and regions?

Organizations API → list-accounts → AssumeRole per account → DescribeRegions → DescribeInstances per region. Alt: Config Aggregator or Resource Explorer.

Q: EC2 within a specific OU?

`list-accounts-for-parent` with OU ID. Recursively for nested OUs. Same assume + describe pattern.

Q: What does 3/3 status check mean?

1: System (AWS infra). 2: Instance (OS). 3: **Attached EBS** (volume health/I/O). Third failure = volumes impaired.

Q: Share AMI with child accounts?

- `modify-image-attribute --launch-permission`
- Org-wide via OrganizationArn
- AWS RAM (recommended)

⚠ Encrypted: share KMS key. Default keys can't be shared — re-encrypt with CMK. Share snapshots too.

Q: Shared AMI instance launches and terminates?

- KMS access denied
- Missing snapshot perms
- IAM role/profile missing
- UserData failures
- Instance limits / subnet mismatch

Q: Diagnose shared AMI issues?

- State Transition Reason
- get-console-output / screenshot
- Test in source account
- CloudTrail for denied errors
- Validate KMS + snapshot sharing

Q: 3-tier architecture steps?

1. VPC, 6 subnets (2 pub + 2 priv app + 2 priv DB), IGW, NAT. **2.** ALB in public. **3.** App ASG in private. **4.** RDS Multi-AZ in private DB. **5.** NACLs, Flow Logs, WAF, CloudWatch.

Q: Get PID?

ps aux | grep name / pidof name / pgrep -f / lsof -i :port / top

---

Q: Check system performance?

- `top/htop` , `vmstat 1` , `iostat -x 1`
- `free -h` , `df -h` , `sar`
- `uptime` , `mpstat -P ALL`

Q: Full permissions?

chmod 777 file / chmod -R 777 folder/
⚠ Security risk. Use 755/644 + chown/chgrp instead.

Q: Soft vs hard link?

Soft: pointer to path (`ln -s`), cross-FS, breaks if deleted. **Hard**: same inode (`ln`), no cross-FS, survives deletion.

Q: Open firewall?

firewall-cmd --permanent --add-port=8080/tcp && --reload # RHEL ufw allow 8080/tcp # Ubuntu iptables -A INPUT -p tcp --dport 8080 -j ACCEPT
On AWS: also configure EC2 Security Group.

Q: netstat -ntlp?

- `-n` numeric, `-t` TCP, `-l` listening, `-p` show PID

Modern: `ss -ntlp`

Q: How do you implement DR in AWS?

- Determine RTO / RPO first.
- **Backup & Restore:** Cross-region snapshots holding state. Slower RTO.
- **Pilot Light:** Database replicated cross-region, infrastructure ready but idle. Data is safe but requires scale-up time.
- **Warm Standby:** A scaled-down version of full prod constantly running.
- **Active-Active:** Multi-region routing with Route 53 latency logic and DynamoDB global syncing for near-zero downtime.

Q: What is RTO and RPO?

- **RTO (Recovery Time Objective):** The maximum acceptable length of downtime. Answers "How quickly must the application be back online?" (e.g., 4 hours).
- **RPO (Recovery Point Objective):** The maximum acceptable tolerance for data loss, measured in *time*. Answers "How much past data can we afford to lose?" (e.g., 15 minutes of transactions).

Q: How do you design multi-region architecture?

- **DNS Layer:** Route 53 health checks and Geolocation/Latency routing policies.
- **Compute Layer:** Independent, duplicated EKS clusters or ASGs in each region. Completely stateless.
- **Data Layer (The Hard Part):** Use Global services like DynamoDB Global Tables or Aurora Global Database. Centralized asynchronous replication.
- **Edge Layer:** CloudFront and Global Accelerator to seamlessly handle user routing over the AWS backbone.

Q: How do you control AWS costs at scale?

- Mandatory infrastructure tagging policies using AWS Organizations SCPs.
- Adopt Savings Plans and Reserved Instances for baseline compute demands.
- Leverage Spot Instances for stateless batch jobs and EKS ephemeral workloads (via Karpenter).
- Use Compute Optimizer to identify over-provisioned resources.
- S3 Object Lifecycle policies to move historical data to Glacier Deep Archive.

Q: What is AWS Well-Architected Framework?

A set of industry best practices guiding the resilient design of cloud workloads across six pillars:

- **Operational Excellence:** Automation and runbooks.
- **Security:** IAM least-privilege, encryption.
- **Reliability:** Multi-AZ, graceful failure mitigation.
- **Performance Efficiency:** Serverless scaling, correct instance types.
- **Cost Optimization:** Resource lifecycle limits and spot instances.
- **Sustainability:** Maximizing utilization to minimize energy footprint.

Q: How do you operate AWS for enterprise workloads?

- Use AWS Control Tower for standardized account provisioning with built-in guardrails.

- Strict immutable infrastructure deployments using Terraform or CDK.
- Centralized observability via dedicated logging accounts pulling CloudTrail, Flow Logs, and centralized Grafana/Datadog.
- Establish an Internal Developer Platform (IDP) with self-service templates prioritizing security by default.

Q: How do you optimize EC2 cost in AWS?

- **Rightsizing:** Downgrade oversized instances using AWS Compute Optimizer data.
- **Architecture:** Migrate workloads to Graviton based (ARM) processors for higher performance at lower cost.
- **Purchasing Models:** Commit to Compute Savings Plans for base load, use Spot Instances for fault-tolerant workers.
- **Automation:** Install scheduled lambda functions to stop dev/test EC2 instances over the weekend.

Q: What is Vertex AI in GCP and where would you use it?

Vertex AI is Google Cloud's premier unified Machine Learning operations (MLOps) platform. Equivalent to AWS SageMaker, it consolidates data engineering, large-language model (LLM) tuning, training, and fully-managed model endpoint deployment under one enterprise umbrella.

Q: How would you deploy GPU workloads on AWS?

- Provision raw heavily-accelerated `p4`, `p5`, or `g5` EC2 instances equipped with Deep Learning AMIs.
- Under Kubernetes/EKS: Exclusively taint GPU nodes, deploy the Nvidia Device DaemonSet plugin, and explicitly mandate `nvidia.com/gpu: 1` limits directly into your container specifications.

Q: Spot vs On-Demand vs Reserved - when to use each

- **Reserved / Savings Plans:** 24/7 baseline operational infrastructure (Production databases, foundational controllers).
- **Spot:** Ephemeral, stateless, interruption-tolerant processing pools (Data pipelines, EKS compute-intensive worker nodes handled via Karpenter).
- **On-Demand:** Sudden, unpredictable workloads possessing zero historical baselines, or instances destined for imminent deletion.

Q: S3 costs tripled - how to investigate and fix

- Interrogate AWS Cost Explorer analyzing exact API vectors (Are costs surging due to storage GBs, external egress Transfer, or runaway LIST mechanisms?).
- Investigate S3 Storage Lens diagnosing potentially millions of lingering, orphaned multi-part uploads accumulating space. Fix via enforcing immediate multipart lifecycle-abort rules.
- Activate Intelligently Tiering comprehensively transitioning untouched analytical archives out into zero-touch Glacier layers.

Q: Your entire production workload runs in a single AWS region, and management asks you to design a multi-region disaster recovery strategy with minimal downtime. How would you approach it?

I would implement a **Warm Standby** or **Active-Active** architecture. 1. Decouple state entirely from compute. 2. Transition databases to **Aurora Global Databases** and DynamoDB Global Tables for sub-second asynchronous cross-region replication. 3. Define all infrastructure strictly in Terraform. 4. Deploy stateless EKS clusters in the secondary region. 5. Route traffic intelligently using Route 53 Latency/Failover records.

Q: Your cluster is facing high resource utilization. How will you optimize it?

- Configure **Karpenter** to consolidate sparsely-utilized large nodes down into smaller, strictly-fitted nodes.
- Re-evaluate Pod **Requests & Limits**; developers frequently over-request resources padding their ego, starving the cluster.
- Implement **VPA (Vertical Pod Autoscaler)** in estimation mode to scientifically gauge what exact requests the pods actually require.

Q: What is Azure App Service?

Azure App Service is a fully managed PaaS for hosting web apps, REST APIs, and mobile backends. It supports multiple runtimes (Node, Python, .NET, Java, PHP). Key features: auto-scaling, custom domains, SSL, deployment slots, and built-in CI/CD integration via GitHub Actions or Azure DevOps. Equivalent concept to AWS Elastic Beanstalk or App Runner.

Q: How do you switch between deployment slots in Azure App Service?

Azure App Service **deployment slots** allow you to host different environments (staging, production) under the same app plan.

- **Swap:** `az webapp deployment slot swap --slot staging --target-slot production` — atomic swap with no downtime
- **Portal:** App Service → Deployment Slots → Swap
- **Why slots matter:** Staging warms up before swap. If prod breaks, swap back instantly. Auto-swap can trigger on successful deployment.
- **Slot-specific settings:** Connection strings marked as "slot setting" do NOT swap — ensures staging DB stays in staging after swap.

🌐 AWS Networking & Load Balancing

Q: How would you make a subnet public or private?

- **Public**: Route table with `0.0.0.0/0 → IGW`
- **Private**: No IGW route. Outbound via NAT GW in public subnet: `0.0.0.0/0 → NAT GW`

Q: Internet Gateway is attached to a subnet or a VPC?

IGW attaches to the **VPC**, not a subnet. A subnet becomes "public" when its route table routes 0.0.0.0/0 to the IGW.

Q: Where are the outgoing rules for a private subnet needing internet?

The rule `0.0.0.0/0 → NAT GW` goes in the **private subnet's route table**. NAT GW lives in the public subnet with its own route to IGW. Flow: private → NAT GW → IGW → internet.

Q: List all choices to connect on-prem to AWS?

- **Site-to-Site VPN**: IPSec over internet; quick, lower cost, variable latency
- **Direct Connect**: Dedicated fiber; consistent low latency, higher cost
- **Direct Connect + VPN**: DX primary, VPN failover
- **Transit Gateway + VPN/DX**: Centralized multi-VPC connectivity
- **VPN CloudHub**: Hub-and-spoke for multiple remote sites

Q: Any other networking options?

- **PrivateLink**: Private service connectivity without internet
- **VPC Peering**: Direct VPC-to-VPC (same/cross-account/region)
- **Transit Gateway**: Hub for many VPCs + on-prem
- **Outposts**: AWS infrastructure in on-prem data centers

Q: Why do you need Transit Gateway when we have VPC Peering?

VPC Peering is **non-transitive**. A↔B and B↔C doesn't mean A↔C. With 10 VPCs = up to 45 peerings. TGW is a central hub — each VPC connects once, TGW handles all routing.

Q: How is Transit Gateway beneficial vs VPC Peering?

- **Scalability**: 1 attachment/VPC vs N*(N-1)/2 peerings
- **Centralized routing**: Single route table
- **Cross-account/region**: Native support
- **On-prem**: Accepts VPN + DX attachments
- **Simplified ops**: One place for all traffic management

Q: Types of load balancers and when to use each?

- **ALB**: Layer 7, HTTP/S. Microservices, path/host routing, WebSocket, gRPC.
- **NLB**: Layer 4, TCP/UDP. Ultra-low latency, static IP, non-HTTP protocols.
- **GWLB**: Layer 3. Inline appliances (firewalls, IDS/IPS).
- **CLB**: Legacy. Avoid for new workloads.

Q: API Gateway vs ALB — when to use each?

- **API GW**: Serverless (Lambda), auth (Cognito/JWT), rate limiting, API versioning. Higher per-request cost.
- **ALB**: Container/EC2 backends, high throughput, WebSocket. Lower cost at scale.

Rule: API GW for public APIs with auth; ALB for internal services and high-volume container traffic.

Q: CloudFormation template for EC2 + ALB?

- VPC, 2+ subnets in different AZs, IGW, routes
- ALB SG (80/443 from 0.0.0.0/0), EC2 SG (from ALB only)
- 3 ALB resources: LoadBalancer, TargetGroup, Listener
- EC2 Instance with UserData
- Outputs for ALB DNS, Parameters for reuse

Q: VPC peering with overlapping CIDRs?

No. Alternatives: PrivateLink, Transit Gateway + NAT, re-address (secondary CIDR), app-layer proxies.

Q: What is VPC endpoint (Interface vs Gateway)?

Gateway Endpoint	Interface Endpoint (PrivateLink)
Modifies the VPC Route Table.	Creates an Elastic Network Interface (ENI) with a private IP.

Only supports S3 and DynamoDB.	Supports almost all AWS services and 3rd-party SaaS.
Free of charge.	Billed hourly + per GB data processed.

Q: How does PrivateLink work?

AWS PrivateLink enables highly secure, direct connections between VPCs, supported AWS services, and on-premises networks **without traversing the public internet**. It achieves this by creating Interface Endpoints (ENIs) directly within your VPC subnets. Traffic flows entirely over the secure AWS global backbone.

Q: What is hybrid connectivity in AWS?

Connecting an on-premises data center securely and transparently into AWS VPCs. Common implementations include Site-to-Site VPN (encrypted over the internet), AWS Direct Connect (dedicated physical fiber connection), AWS Client VPN for remote users, and managing routing centrally via Transit Gateway.

Q: Difference between Direct Connect and VPN?

Feature	Direct Connect (DX)	Site-to-Site VPN
Medium	Dedicated physical fiber link	Over the public internet
Bandwidth	Consistent, 1-100 Gbps	Variable, max ~1.25 Gbps per tunnel
Latency	Low and reliable	Variable (internet weather)
Lead Time	Weeks/Months (requires colocation)	Minutes to deploy

---

Q: How does ALB handle millions of requests?

ALB inherently scales automatically by adding processing nodes directly behind the scenes. However, sudden instantaneous spikes can overwhelm it before it scales. For known, massive traffic events (e.g., Black Friday launch), you must submit a ticket to AWS Support for **pre-warming** to ensure sufficient ALB capacity is pre-provisioned.

Q: What is connection draining (deregistration delay)?

When an EC2 instance in a Target Group becomes unhealthy or is scaling down, **Connection Draining** prevents the ALB from severing active connections immediately. The ALB stops sending *new* requests but waits for *in-flight* requests to complete up to a configured threshold (default 300 seconds) before fully deregistering the instance.

Q: Difference between Security Group vs NACL

Security Group (SG)	Network ACL (NACL)
Operates at the **Instance / ENI** layer.	Operates at the **Subnet** layer.
Stateful: Inbound traffic allowed auto-allows return traffic.	**Stateless:** Return traffic must be explicitly allowed.
Default: Deny all inbound, Allow all outbound.	Default: Allow all inbound, Allow all outbound.
Only supports ALLOW rules.	Supports both ALLOW and DENY rules.

Q: Explain end-to-end request flow: 👉 Internet → AWS → Application

1. Request: User types URL. **2. DNS:** Route 53 resolves domain to CloudFront. **3. CDN:** CloudFront serves static cache or proxies to backend. **4. Security:** WAF inspects traffic at CloudFront. **5. Entry:** Request hits Internet Gateway. **6. Proxy:** Reaches ALB in public subnet. **7. App:** ALB routes to EC2/EKS in private subnet. **8. Data:** App queries RDS in isolated DB subnet.

Q: What are the types of Load Balancers in AWS and how do they work?

- **Application Load Balancer (ALB):** Layer 7. Deeply inspects HTTP/S protocols. Routes based on path, host, or HTTP headers. Ideal for microservices.
- **Network Load Balancer (NLB):** Layer 4. Handles TCP/UDP traffic. Ultra-low latency, handles millions of requests/sec instantly, supports static IPs.
- **Gateway Load Balancer (GWLB):** Layer 3/4. Designed specifically to run and scale inline 3rd-party virtual appliances (like Palo Alto firewalls).

Q: Security Groups vs NACLs - and the follow-up question 80% miss

The core disparity lies in persistent state mapping. SGs operate statelessly at the ENI boundary (inbound ALLOW implicitly guarantees symmetric outbound return traffic). A typical 80% trap-failure occurs when engineers write a strict NACL inbound rule expecting an HTTP response to function, critically failing to explicitly map an asymmetric outbound NACL ALLOW handling the broad Ephemeral internal port range (1024-65535).

Q: VPC Peering vs Transit Gateway decision framework

Use **VPC Peering** exclusively for isolated, minor setups involving <5 VPCs commanding extreme bandwidth with absolutely zero necessity for lateral hub routing (totally free traffic routing). Command **Transit Gateway (TGW)** immediately for enterprise footprints. TGW solves explosive non-transitive mesh complexity, unifies Direct Connect attachments gracefully, and centralizes security inspection appliances structurally.

Q: Why does AWS reserve 5 IP addresses in every subnet, and what are they used for?

- `.0` structurally designates the fundamental Network address identifying the subnet.
- `.1` intrinsically allocated strictly to the VPC hardware routing appliance boundary.
- `.2` heavily utilized exclusively by the internal Amazon DNS (Route 53 Resolver endpoint).
- `.3` systemically reserved for nebulous future AWS feature additions.
- `.255` systematically represents the ubiquitous Network Broadcast frame limit capability.

🔒 AWS IAM & Security

Q: Security best practices across each area?

- **IAM**: Least privilege, no root, MFA, OIDC federation
- **Network**: Private subnets, SGs > NACLs, PrivateLink
- **Data**: KMS at rest, TLS in transit, Secrets Manager
- **Compute**: IMDSv2, SSM instead of SSH
- **Container**: Non-root, read-only FS, ECR scanning
- **Logging**: CloudTrail all regions, Flow Logs, GuardDuty, Security Hub
- **Patching**: SSM Patch Manager, AMI baking pipelines

Q: Bare minimum to keep cloud secure?

- CloudTrail + GuardDuty everywhere
- MFA on all humans, especially root
- No public S3 — block at account level
- No long-lived keys — use OIDC/roles
- VPC Flow Logs
- Encrypt all storage at rest
- SCPs via Organizations

Q: How to extend access without compromising security?

IAM OIDC Federation — federate with IdP (Okta, Azure AD, GitHub Actions). Users/workloads assume roles with short-lived tokens. No static credentials, full auditability.

Q: Do you know what service that is?

AWS IAM Identity Center (SSO) for humans, **OIDC-based IAM roles** for workloads (GitHub Actions OIDC, EKS IRSA).

Q: Have you worked with cross-account KMS?

Yes — sharing KMS keys across accounts for encrypted AMIs, S3, Secrets Manager.

Q: How do you facilitate that connectivity?

KMS key in **source account** must have a key policy granting destination account permission to use it.

Q: What configuration is required?

- **Key Policy** (source): Allow destination principal for kms:Decrypt, kms:GenerateDataKey, etc.
- **IAM Policy** (destination): Role/user must also have IAM policy permitting KMS actions on that key ARN.

Both sides must grant — key policy alone isn't enough without matching IAM policy.

Q: Do you take care of Docker image security?

Yes — ECR scanning (Trivy-based) + optionally Snyk. Images with critical/high CVEs blocked from promotion.

Q: Are K8s Secrets really secrets?

No — just **base64-encoded, not encrypted**. Anyone with `kubectl get secret` access decodes instantly. Stored in etcd which may not be encrypted by default.

Q: How to make K8s Secrets more secure?

- Enable **etcd encryption at rest**
- **External Secrets Operator** + Vault/Secrets Manager
- **Sealed Secrets** for GitOps
- Strict RBAC on secret access
- At client: Vault + K8s auth + Vault Agent Injector — secrets injected as files, never stored in K8s

---

Q: Where are secrets stored?

In Secrets Manager or Vault. Fetched dynamically via `aws_secretsmanager_secret_version` data source. Never hardcoded in tfvars or state.

Q: How ensure secrets aren't exposed?

- .gitignore sensitive tfvars
- Pipeline injects from Vault/Secrets Manager
- S3 state encrypted with KMS + IAM-restricted
- State never stored locally
- State access audited via CloudTrail

Q: Multi-account IAM role via CloudFormation?

- **StackSets**: Deploy across accounts/OUs, auto-deploy to new accounts
- **Service-Managed**: Organizations trusted access, no manual roles
- **Self-Managed**: Requires admin + execution roles (bootstrap)

Q: Managing secrets in Ansible?

- Ansible Vault (AES-256)
- External: Vault, Secrets Manager via lookup
- Env vars: lookup('env', ...)

Q: Securely run playbooks with secrets?

- --vault-password-file (chmod 600)
- --ask-vault-pass
- Vault IDs per env
- CI/CD: pipeline secret → temp file
- Tower/AWX: encrypted creds
- no_log: true

Q: How does AWS IAM evaluate policies?

- **Explicit Deny** overwrites everything.
- **Explicit Allow** grants access if there's no Deny.
- **Default Deny** (Implicit Deny) applies if neither exists.
- **Context order:** Organizations SCP > Resource-based Policy > Identity-based Policy > IAM Permissions Boundary > Session Policy.

Q: What is policy evaluation logic order?

The precise step-by-step logic is:

1. By default, all requests are implicitly denied.
2. Evaluate all applicable policies (SCP, Identity, Resource, Boundary, Session).
3. If **any** applicable policy has an `Effect: Deny`, the request is definitively denied.
4. If no Deny exists, but an `Effect: Allow` exists in the applicable policies (considering boundaries), it is allowed.
5. If no Allow matches, the default implicit deny takes over.

Q: Difference between identity-based and resource-based policies?

Identity-based	Resource-based
Attached to IAM User, Group, or Role.	Attached to a resource (S3 bucket, SQS, KMS key).
Specifies what the identity can do.	Specifies who can access the resource and how.
Cannot grant cross-account access on its own.	Crucial for granting cross-account access.

Q: How does AWS STS work internally?

AWS Security Token Service (STS) issues short-lived, temporary credentials (Access Key, Secret Key, Session Token). Flow:

- Client calls `AssumeRole`.
- STS verifies the requester's identity policy and the target role's trust policy.
- If both allow, STS dynamically generates temporary credentials valid for 15 mins to 12 hours.
- These credentials are automatically rotated and expire.

Q: How do you implement cross-account access securely?

- Never share long-lived IAM user keys across accounts.
- Create an IAM Role in the **Target Account** with a *Trust Policy* allowing the Source Account to assume it.
- In the **Source Account**, attach an IAM Policy to an entity allowing `sts:AssumeRole` to the Target ARN.
- **Crucial:** Use `ExternalId` in the Trust Policy to prevent the Confused Deputy problem.

Q: What is AWS Organizations and SCP?

- **AWS Organizations**: Service to centrally manage and govern multiple AWS accounts, enabling consolidated billing and logical grouping (OUs).
- **Service Control Policies (SCPs)**: Guardrails applied at the Root, OU, or Account level. They define the *maximum available permissions* globally across the accounts, regardless of how permissive the IAM policies inside those accounts are.

Q: How do Service Control Policies work?

SCPs act as a filter. They **do not grant permissions**. Even if an IAM Administrator in a child account has `AdministratorAccess`, if an SCP applied to their account denies `s3:*`, that admin cannot access S3. The SCP intercepts the API call before standard IAM evaluation.

Q: How do you enforce security guardrails in AWS?

- **Preventive Guardrails**: Use AWS Organizations SCPs to block prohibited actions globally (e.g., denying creation of VPC internet gateways).
- **Proactive Guardrails**: IAM Permissions Boundaries to limit the maximum permissions developers can grant to roles they create.
- **Detective Guardrails**: AWS Config rules to flag or auto-remediate non-compliant resources (e.g., untagged EC2s, public S3 buckets).

Q: How do you secure S3 buckets at scale?

- Enable **Block Public Access** at the Account or Organization level.
- Enforce encryption at rest (SSE-KMS) via Bucket Policies.
- Use Macie for automated PII discovery.
- Enable Server Access Logging / CloudTrail data events to monitor access patterns.
- Implement strict IAM Least Privilege and Resource-based Bucket Policies.

Q: How do you prevent data exfiltration from S3?

- Ensure traffic to S3 stays private via **VPC Gateway Endpoints**.
- Apply a VPC Endpoint Policy restricting access exclusively to your own specific buckets.
- Create a Bucket Policy that only accepts traffic from that specific VPC Endpoint.
- Use SCPs to prevent IAM users from creating bucket policies that share data outside the Organization.

Q: What is AWS Shield vs WAF?

AWS Shield	AWS WAF
Layer 3 and 4 DDoS protection.	Layer 7 HTTP/S application firewall.
Standard (free) guards against SYN floods/UDP reflection.	Protects against SQL injection, XSS, bad bots.
Advanced (paid) offers custom mitigation and billing protection.	Relies on customizable rule sets and managed rules.

Q: How does AWS WAF protect applications?

WAF attaches to ALBs, API Gateway, or CloudFront. It inspects incoming HTTP/S requests against configured Web ACLs (rules). It can block or count requests based on geo-location, IP range, rate limiting, suspected SQL injection patterns, or custom header matching before the traffic reaches the backend.

Q: How do you detect compromised IAM credentials?

- Enable **AWS GuardDuty** — it uses ML algorithms to detect anomalous API activity (e.g., login from an unexpected geographical location, mass resource deletion, or disabling CloudTrail).
- Configure AWS Budgets to alert on sudden cost spikes (crypto mining behavior).
- Enforce MFA dynamically using IAM condition keys.

Q: How would you restrict pod-to-pod communication using Calico Network Policies?

You establish a Zero-Trust namespace boundary. Deploy a `NetworkPolicy` mapping to specific pod selectors that institutes a **Default Deny** for all ingress and egress. Then, add granular exceptions strictly authorizing traffic solely between Pods harboring the precise explicit matching labels (e.g., allow `db-pod` ingress only from `backend-pod`).

Q: How do you secure AWS credentials in production environments?

Zero static credentials. Environments rely completely on intrinsic workload identities. Applications within EKS employ IRSA. EC2 instances employ Instance Profiles. GitHub Actions pipelines implement IAM OIDC Federation. At no point does a human generate or store an Access Key geometrically diminishing exfiltration threats.

Q: How do you implement least privilege access across environments?

- Strict Multi-Account Organizational segmentation (Dev, Stg, Prod Accounts).

- AWS IAM Service Control Policies (SCPs) placing hard ceilings universally (e.g. denying Region usage, demanding encryption).
- Production developer access functionally suppressed to Read-Only; all mutations rigorously forced manually through the CI/CD principal pipeline.

Q: What are OWASP Top 10 aligned security rules?

Categorical rules shielding applications against globally profound vulnerabilities: Injection (SQLi), Flawed Access Controls, SSRF (Server-Side Request Forgery), and Cross-Site Scripting (XSS). In AWS, deploying the Managed Rule Groups identically mapping to the OWASP Top 10 straight onto the WAF eliminates superficial application exploitation angles automatically.

☸ Kubernetes & Containers

Q: Any specific reason you'd use EKS and not ECS?

EKS gives the full Kubernetes API — portable, vendor-neutral, ecosystem-rich. When the org already uses K8s tooling (Helm, ArgoCD, Kyverno, Prometheus), EKS is the natural fit. Supports multi-cloud portability.

Q: Why do you think ECS won't be a good choice in that scenario?

ECS is AWS-proprietary. For advanced scheduling, custom controllers, CRDs, or GitOps workflows, ECS falls short. No Helm, no ArgoCD, no KEDA, no Istio natively.

Q: Don't you think ECS also provides that kind of stuff which EKS does?

ECS has improved — Fargate, service discovery, task-level IAM. But it lacks K8s control plane extensibility. No custom operators, admission webhooks, or CRDs. Ecosystem depth isn't comparable.

Q: Scenario where ECS will be better suited compared to EKS?

- Simple microservices with no K8s expertise on team
- Teams wanting AWS-managed everything with minimal overhead
- Fargate workloads — no node management
- Smaller orgs without a platform team

Q: Which one is cheaper — EKS or ECS?

ECS (especially Fargate) is generally cheaper for smaller workloads. EKS charges $0.10/hr per cluster (~$72/mo) plus node costs. ECS has no control plane fee.

Q: Why is ECS cheaper?

No control plane cost, no etcd overhead. Fargate = pay only for usage, no idle node capacity. For sporadic/low-traffic workloads, significantly cheaper.

Q: Do you create Docker images or get them from developers?

Both — I own Dockerfiles for platform services. For app services, developers own Dockerfiles but I enforce standards through pipeline gates and base image policies.

Q: Docker best practices for image creation?

- Minimal base (alpine, distroless)
- Multi-stage builds
- Non-root user
- Pin versions — never `latest` in prod
- .dockerignore
- Minimize layers
- No secrets in image or ENV
- Immutable — use env vars/mounted configs

Q: Where do you host Docker images?

AWS ECR — private repos, IAM integrated, native to EKS/ECS.

Q: How do you control number of images in ECR?

ECR **Lifecycle Policies** — rules like "keep last 10 tagged" or "expire untagged >7 days." Per repository.

Q: Practices to ensure ECR images aren't orphaned?

- Lifecycle policies for untagged
- Tag with Git SHA + semver
- Only scanned images promoted to prod
- Separate repos per env with promotion pipelines
- Periodic audits via ECR API

Q: How ensure Docker image works same on dev machine and EKS?

- Consistent base images via internal registry
- `--platform linux/amd64` (important for Apple Silicon)
- Multi-arch builds (`buildx`)
- Pipeline rebuilds from source — local image ≠ deployed image
- Integration tests against pipeline-built image

Q: How ensure developer scans locally AND pipeline has control?

- `make scan` target with Trivy locally
- Pipeline mandatory scanning gate
- Pre-commit hooks with Hadolint
- Policy: devs validate locally; pipeline enforces

Q: How to reduce Docker image size?

- Multi-stage builds
- Distroless/alpine base
- Remove build tools in same RUN layer
- No unnecessary packages
- `--no-install-recommends`
- `docker history` + `dive` to find large layers

Q: How do you update EKS clusters?

- Update **control plane** first (AWS manages)
- Rolling update **managed node groups**
- Update **add-ons** (CoreDNS, kube-proxy, VPC CNI)
- Health checks at each step

Q: How to ensure zero downtime during EKS upgrade?

- **PodDisruptionBudgets** on all critical workloads
- Rolling node group upgrades (new up, old drain)
- 2+ replicas with anti-affinity across AZs
- Smoke test after control plane before node upgrade
- Test on staging first

Q: If application stops working, what are next steps?

- `kubectl get pods`, `describe pod`
- `kubectl logs --previous`
- `kubectl get events --sort-by='.lastTimestamp'`
- Check resource limits (OOMKilled, CPU throttle)
- `kubectl rollout history` → `rollout undo`
- Check node health

Q: Have you worked with stateful applications on K8s?

Yes — Redis cluster as StatefulSet with PVs (EBS CSI driver) and headless service for stable DNS.

Q: Why deploy it on Kubernetes?

Application team wanted operational consistency — all services via same GitOps pipeline. In hindsight, ElastiCache would've been simpler and more operationally sound.

Q: But you didn't question them — as DevOps you should push back?

Agreed. Should have done cost-benefit analysis: EKS-hosted Redis (operational overhead, storage, backup) vs ElastiCache (managed, auto-failover, minimal ops). Platform engineers should proactively challenge, not just execute.

Q: Does K8s support blue-green deployment?

- No native object, but use standard primitives:
- Deploy green (new) with label `version: green`
- Blue (current) has `version: blue`
- Service selector → blue. Validate green. Switch to green.
- With ArgoCD + Argo Rollouts: automated BlueGreen strategy

Q: Kubernetes best practices?

- Resource requests + limits on all containers
- PodDisruptionBudgets
- NetworkPolicies
- RBAC — least-privilege SAs

- Readiness + liveness probes
- Namespaces for isolation
- Non-root containers
- HPA/VPA for autoscaling
- Consistent labels/annotations
- ConfigMaps/Secrets, not baked configs

Q: K8s troubleshooting scenarios?

- **CrashLoopBackOff**: Bad config, missing env. `logs --previous`
- **ImagePullBackOff**: ECR auth, wrong tag. Check IRSA
- **Pending**: Insufficient resources, PVC unbound, selector mismatch
- **OOMKilled**: Memory limit exceeded
- **DNS failure**: CoreDNS overloaded
- **Service unreachable**: Wrong selector, port mismatch, NetworkPolicy

---

Q: Why do you need Ingress?

Without Ingress, each externally-accessible service needs its own LB — expensive and messy. Ingress consolidates external traffic through a single entry with host/path routing.

Q: How is Ingress different from Services?

Service: Internal cluster networking + LB between pods. **Ingress**: L7 routing in front of Services — host/path routing, SSL termination, external traffic management.

Q: What tool for deploying applications?

ArgoCD for GitOps-based K8s deployments.

Q: Only ArgoCD or Helm + ArgoCD?

Helm + ArgoCD. ArgoCD manages GitOps sync; Helm handles templating and packaging.

Q: Why add Helm complexity?

Helm solves **environment-specific configuration** — single chart with templated values, overridden per env. Versioning + rollback of entire stacks. Community chart ecosystem saves time.

Q: Experience with probes?

Hands-on with all three types for production workloads.

Q: How are probes different from each other?

- **Liveness**: Is container alive? Fails → **restart**. Deadlock detection.
- **Readiness**: Can it serve traffic? Fails → **removed from endpoints**. No restart.
- **Startup**: Done starting? Disables other probes until pass. For slow starters.

Q: Why are probes needed?

Without them, K8s sends traffic to uninitialized pods (errors) or keeps sending to deadlocked pods (failures). Configure realistic `initialDelaySeconds`, `failureThreshold`, `periodSeconds` from actual benchmarks.

Q: What are ECS and ECR?

ECS: Managed orchestration (EC2/Fargate). Concepts: Task Defs → Tasks → Services. **ECR**: Managed Docker registry with scanning, lifecycle policies, KMS encryption.

Q: How to create and push Docker images to ECR?

1. Create Dockerfile 2. aws ecr create-repository --repository-name my-app 3. aws ecr get-login-password | docker login ... 4. docker build -t my-app . 5. docker tag my-app:latest <acct>.dkr.ecr.<region>.amazonaws.com/my-app:latest 6. docker push ...

Q: Optimize a 450MB Docker image?

- Alpine/distroless base
- Multi-stage builds (400MB → <50MB)
- Combine RUN, .dockerignore
- --no-install-recommends, production deps only
- Layer ordering for cache

Q: Docker vs Kubernetes in orchestration?

Docker: Runtime + toolset. Builds/runs containers. **K8s**: Orchestration at scale — scheduling, scaling, healing, discovery, updates. Complementary.

Q: What is a pod in Kubernetes?

Smallest unit — 1+ containers sharing network (same IP/localhost), storage, lifecycle. Ephemeral. Use Deployments to manage.

Q: Container vs Pod?

Container = single process. Pod = wrapper for 1+ containers with shared networking + volumes. K8s manages at pod level. Scale by adding pods.

Q: Load balancing in Kubernetes?

- ClusterIP (default, iptables/IPVS)
- NodePort (static port per node)
- LoadBalancer (cloud LB)
- Ingress (L7 routing)
- kube-proxy maintains rules

Q: What is HPA (Horizontal Pod Autoscaler)?

HPA dynamically adjusts the raw number of replicas deployed within a Kubernetes Deployment, ReplicaSet, or StatefulSet based on observed metrics (traditionally CPU utilization spikes passing a set threshold). It ensures peak load is serviced cleanly, then aggressively scales down instances afterward to suppress expenditure.

Q: Explain the Docker commands used inside a Dockerfile

- `FROM` : Pulls down the base operating system image (Alpine/Ubuntu).
- `COPY` : Ingests specific files from the host repository into the container filesystem.
- `RUN` : Executes arbitrary terminal commands sequentially *during the build phase/layer construction* (e.g., npm install).
- `EXPOSE` : Purely documentation—dictates the intended default listening ports.
- `CMD/ENTRYPOINT` : Dictates the exact command executed when the completed container instance actually spins up.

Q: What is Pod Intent in Kubernetes?

Pod Intent refers to the *Desired State* conceptually defined in a high-level controller (like a Deployment or StatefulSet). Instead of managing Pods manually, you declare your intent (e.g. "Ensure exactly 3 replicas of this image run at all times"). The Kubernetes Control Plane continuously evaluates the cluster state and actively reconciles the actual state to match your declared intent.

Q: How does Rollback Deployment work internally?

When you trigger `kubectl rollout undo`, Kubernetes does not physically build anything. The Deployment controller simply drops the replica count of the current ReplicaSet to 0, and restores the replica count of the previous version's ReplicaSet (which was retained in the cluster history) back to the desired number. This makes rollbacks incredibly fast.

Q: Difference between HPA, KEDA, and Karpenter?

- **HPA (Horizontal Pod Autoscaler):** Native K8s resource. Scales *Pods* up/down mostly based on internal resource utilization (CPU/Memory).
- **KEDA:** Event-driven autoscaler. Scales *Pods* up/down based on external system metrics (e.g. RabbitMQ queue length, Kafka lag). Can scale to 0.
- **Karpenter:** Node autoscaler. Directly provisions raw EC2 *Nodes* into the cluster in seconds when pods are waiting to schedule, bypassing standard ASGs.

Q: Why do we still need HPA if Karpenter is being used?

Because they scale entirely different dimensions. HPA observes application load and requests more **Pods**. Karpenter observes cluster capacity and requests more **Nodes**. Without HPA, Kubernetes won't know it needs more application instances. Karpenter purely reacts when HPA creates pods that cannot fit on existing hardware.

Q: How does Service A communicate with Service B inside Kubernetes?

Via internal DNS managed by CoreDNS. Service A requests `http://service-b.namespace.svc.cluster.local`. CoreDNS resolves this to Service B's virtual ClusterIP. The request is intercepted by `kube-proxy` (via iptables/IPVS) sitting on every node, which transparently load-balances the traffic to the healthy backing Pods representing Service B.

Q: How does a pod securely connect to S3 without hardcoding credentials?

- Use **IAM Roles for Service Accounts (IRSA)** in EKS.
- Associate an AWS IAM Role via an OIDC provider with a specific Kubernetes ServiceAccount.
- Attach the ServiceAccount to the Pod. EKS automatically injects a projected token file and environment variables into the Pod.
- The AWS SDK uses that token to dynamically acquire temporary STS credentials.

Q: How do you implement RBAC policies in Kubernetes?

Define a **Role** (namespace-scoped) or **ClusterRole** (cluster-scoped) enumerating authorized API `verbs` (get, list, create) against discrete `resources` (pods, secrets). Subsequently, execute a **RoleBinding** or **ClusterRoleBinding** structurally linking that Role to distinct Subjects (User, Group, or ServiceAccount).

Q: What are multiple tainted nodes in Kubernetes and when are they used?

Taints are protective wards painted onto Nodes preventing general pods from scheduling there. For instance, applying `sku=gpu:NoSchedule` on expensive GPU EC2 instances ensures mundane web-server pods don't cannibalize ML resources. Only machine-learning pods carrying the corresponding explicit *Toleration* can bypass the taint.

Q: If I want to run two applications in the same EKS cluster but keep them isolated, how would you design it?

- Deploy them into radically separate **Namespaces**.
- Apply strict **Network Policies** to outright terminate cross-namespace network communication.
- Establish strict **RBAC** so developers of App A cannot interrogate App B's secrets.
- Optionally force hard physical isolation using node Taints, labeling specific worker nodes exclusively for App A.

Q: What are dedicated nodes in Kubernetes?

Nodes isolated administratively for specialized workloads, distinct hardware requirements, or rigorous compliance separations. Orchestrated via NodeAffinity (forcing pods to those nodes) paired with Taints (repulsing extraneous pods away from those nodes).

Q: What is a CSI Driver (Container Storage Interface)?

The standardized abstraction layer enabling external storage vendors (AWS EBS, EFS, NetApp) to plug into Kubernetes seamlessly. It replaces deprecated "in-tree" storage plugin architectures, allowing independent capability updates (e.g., EBS volume snapshotting) without updating the core Kubernetes binary.

Q: Which Storage Classes are commonly used in Kubernetes?

- `gp3` (EBS): Ideal for single-pod stateful sets demanding robust performance block storage (ReadWriteOnce).
- `efs-sc` (EFS): Elastic File System optimized for multi-pod synchronized parallel access (ReadWriteMany).
- `io2` : Mission-critical, ultra-low latency databases.

Q: What would you do if Amazon EKS runs out of IP addresses? How can you resolve it?

By default, EKS allocates a unique VPC IP to every pod. If the subnet exhausts IPs via IP exhaustion:

- Enable **Prefix Delegation** on the VPC CNI (allocates /28 blocks instead of dragging individual IPs).
- Utilize **Custom Networking** to tether secondary, massive CGNAT subnets purely for pod assignments.
- Migrate the cluster permanently to an IPv6 framework.

Q: What is an IngressClass in Kubernetes, and which ingress controller is typically used with Amazon EKS?

`IngressClass` conceptually acts as a routing tag, directing an Ingress resource precisely to its appropriate Controller when multiple controllers execute in one cluster. In EKS, the industry-standard is the **AWS Load Balancer Controller** (which dynamically spawns native AWS ALBs) alongside NGINX ingress.

Q: If you have 4 microservices, would you deploy them in 4 separate pods or in a single pod? Why?

4 separate pods. Combining independent services into one pod violates fundamental decoupling. Separate pods guarantee total fault isolation (Service A crashing won't take down B), permit autonomous scaling heuristics (HPA), allow disparate deployments, and enforce distinct resource limits. Sidecars (logs/proxies) are the only valid multi-container pattern.

Q: CrashLoopBackOff on EKS - step-by-step investigation

1. `kubectl describe pod` evaluating Events (Detecting immediate Liveness probe timeout cascades or critical OOMKilled evictions).
2. `kubectl logs <pod> --previous` structurally surfacing exactly what internal panic immediately preceded the previous crash instance.
3. Verifying explicit configuration absence via missing explicit Secrets/ConfigMaps.

Q: Pods in one Kubernetes namespace can communicate internally, but pods in another namespace cannot reach them even though services are exposed. How would you troubleshoot this networking issue?

The most probable cause is a highly restrictive **NetworkPolicy** operating within the destination namespace preventing cross-namespace ingress. I would execute `kubectl get networkpolicy -n `, and verify if a default-deny policy exists without an explicit exception allowing ingress from the source namespace's labels.

Q: Your CI/CD pipeline successfully builds and deploys a container image, but the Kubernetes deployment keeps pulling the old image version. What could be causing this?

Usually caused by recycling the exact same `:latest` or identical static image tag. Kubernetes caches images locally on nodes. If the pod specification's `imagePullPolicy` defaults to `IfNotPresent` (which it does for non-latest tags), the Kubelet will simply boot the stale cached image. **Fix:** Always use unique immutable tags (like Git SHAs) per build, or force `imagePullPolicy: Always`.

Q: Your application is under heavy traffic but Kubernetes HPA is not scaling pods even though CPU usage is above the configured threshold. How would you debug this?

First, verify if the **Metrics Server** is installed and functioning (`kubectl top pods`). Second, verify that the deployment's Pod template actually specifies **CPU Resource Requests**; HPA literally cannot calculate a utilization percentage if there is no baseline request to divide against. If both exist, examine `kubectl describe hpa` for specific controller error events.

Q: New worker nodes are successfully joining an EKS cluster, but pods remain stuck in Pending state. What could be the possible reasons?

1. Exhaustion of VPC IP addresses (the AWS VPC CNI cannot assign secondary IPs to the new nodes).
2. The new nodes possess specific **Taints** (e.g., GPU nodes) that the pods lack Tolerations for.
3. Strict **PodAntiAffinity** rules or `nodeSelectors` aggressively preventing scheduling on those particular newly joined instance types.

Q: A pod is in CrashLoopBackOff state. How will you troubleshoot step by step?

1. Check pod status: `kubectl describe pod ` -> Look at 'Events' for Liveness/Readiness probe failures or OOMKilled errors.
2. Check current logs: `kubectl logs `.
3. Check previous crashed instance logs: `kubectl logs --previous`.
4. Check configurations: Missing Environment variables, missing ConfigMaps, or incorrect secrets causing the entrypoint script to exit 1.

Q: Your application is not accessible via Ingress. What could be the issue?

Work backwards from internet to pod:
1. **DNS/LB:** Does the Route53 domain point to the ALB target group/Ingress LB IP?
2. **Ingress Object:** `kubectl describe ingress` - check if the host and path rules map exactly to the target Service.
3. **Service:** `kubectl describe svc` - check if it has `Endpoints`. If Endpoints are empty, the Service Selector's labels do not match the Pod labels.
4. **Pods:** Are the pods actually legally healthy (passing Readiness probes)?

Q: Pods are not getting scheduled on nodes. What will you check?

Run `kubectl describe pod `. The Scheduler events will explicitly print why it failed. Common causes:
1. **Insufficient Resources:** No node has enough free CPU/Memory to satisfy the pod's `requests`.
2. **Taints/Tolerations:** Nodes are tainted (e.g., `NoSchedule`) and the pod lacks the matched toleration.
3. **NodeAffinity/Selector:** The pod is demanding an explicit label (e.g., `disktype=ssd`) that no healthy node possesses.

Q: One node in the cluster is NotReady. How do you investigate and fix it?

1. `kubectl describe node ` -> check 'Conditions' (MemoryPressure, DiskPressure, PIDPressure).
2. If unhelpful, SSH or Session Manager into the node. Check Kubelet service status (`systemctl status kubelet`) and logs (`journalctl -u kubelet`).
3. Common infrastructure causes: Node ran out of physical disk space `/var/lib/docker` (DiskPressure), or the underlying EC2 instance was abruptly terminated/network partitioned.

Q: Application inside pod is slow despite sufficient resources. Where will you debug?

If CPU/Mem are not throttling, the bottleneck is external IO:
1. **Database Latency:** Slow queries, missing indexes, or connection pool exhaustion.
2. **Network Latency:** CoreDNS lookup delays, or excessively traversing cross-AZ network hops.
3. **Disk IOPS limits:** The underlying EBS volume attached to the node hit its burst IOPS ceiling, causing the kernel to physically sleep the app's threads.

Q: You need to scale applications automatically. How does HPA work internally?

The Horizontal Pod Autoscaler runs as a control loop (default 15s). It queries the **Metrics Server** API `metrics.k8s.io` to fetch the current CPU/Memory usage of all pods in the target Deployment. It compares the current usage average

against your target percentage (e.g., Target 80%). If reality > target, it algebraically computes the required replica count, and directly modifies the Deployment's `.spec.replicas` up to `maxReplicas`.

Q: Config changes are made, but pods are not picking up updates. Why?

If a `ConfigMap` or `Secret` is updated, pods using them as **Environment Variables** will NEVER pick up the changes unless the pod is physically restarted (deleted/recreated). If mounted as a **Volume**, the symlinked files update dynamically after a cache delay (~1 minute), but the application binary itself must be programmed to "hot-reload" files (like NGINX or Promtail), otherwise it stays ignorant of the file change.

Q: You need to secure your Kubernetes cluster. What best practices will you follow?

- **RBAC:** Least-privilege roles, no wildcards.
- **Network Policies:** Default-deny ingress/egress globally, map strict explicit connections.
- **Image Scanning:** Never deploy `:latest`. Enforce Mutating Admission Webhooks (like Kyverno) absolutely rejecting images exhibiting CRITICAL CVEs.
- **Node Security:** EKS private control-plane endpoints, nodes living solely in private subnets, SSM instead of SSH.

Q: Difference between RUN, CMD, and ENTRYPOINT?

- **RUN:** Executes commands directly inside the container *during the image build phase* (e.g., `RUN apt-get install`).
- **ENTRYPOINT:** The immutable executable configured to run when the container starts. Difficult to override at runtime.
- **CMD:** Default arguments passed directly into the `ENTRYPOINT`. Easily overridden dynamically when running `docker run  `.

Q: How do you pass environment variables to containers?

1. **Locally:** Inject via `docker run -e key=value ...` or passing a `--env-file .env` flag.
2. **Docker Compose:** Defining them beneath the `environment:` stanza or sourcing from an `env_file`.
3. **Kubernetes:** Explicitly mapping them identically inside the Pod YAML spec (`env:` array) or mapping entire `ConfigMaps` and `Secrets` holistically via `envFrom:`.

Q: Difference between ARG and ENV?

ARG	ENV
Exists strictly *during the build phase* only.	Persists natively inside the final running container.
Passed dynamically via `docker build --build-arg`.	Passed dynamically via `docker run -e`.
Cannot be interrogated by the running application.	Read easily by the application via standard OS environment variables.

Q: What is a multi-stage Docker build?

A paradigm employing multiple `FROM` statements situated inside a single `Dockerfile`. The primary (builder) stage incorporates heavy compilers (like the Go toolchain or Node `devDependencies`) to violently compile identical source

code. The subsequent final stage imports a pristine, lightweight baseline OS inherently copying *only* the compiled binary executable from the initial stage, violently slashing the final image footprint from GBs to MBs.

Q: How do you reduce Docker image size?

- Implement **multi-stage builds** to strip compilers.
- Adopt minimal foundational base images (e.g., `alpine`, `distroless`, or `scratch`).
- Chain `RUN` instructions functionally utilizing `&&` so package managers solely emit a single discrete layer.
- Systematically eradicate cached APK/APT repository artifacts post-installation within identically the same layer (`rm -rf /var/lib/apt/lists/*`).
- Write a draconian `.dockerignore` file.

Q: How does Docker build cache work?

Docker constructs images progressively as a succession of individually immutable layers. Upon invocation, Docker checks whether the corresponding instruction identically matches a securely cached layer previously built. If yes, it leverages the cache instantly spanning seconds instead of minutes. **Crucial caveat:** The exact instant one line changes (or a `COPY` instruction registers altered file checksums), Docker intrinsically invalidates the cache universally for *all* subsequent downstream layers.

Q: How do you optimize Dockerfile for CI/CD speed?

Intelligently structure layers symmetrically to maximize Cache Hits.
1. Always `COPY package.json` distinctively *before* copying universally fluctuating source code.
2. Execute `RUN npm install` purely against the `package.json`.
3. *Finally* `COPY src/ .`.
This guarantees that rapidly iterating developers mutating source code files will **never** needlessly trigger dependency re-installations.

Q: How do you handle secrets in Docker builds?

Never inject keys via standard `ARG` or `ENV` because they universally persist forever irreversibly baked into the image history manifest (`docker history`).
Instead, utilize Docker's `BuildKit` secrets mechanism: invoke `RUN --mount=type=secret,id=my_secret` to temporarily expose the secret singularly within that exact localized step natively without permanently embedding it into subsequent filesystem layers.

Q: How do you debug a container that exits immediately?

If the PID 1 process executes fully and successfully returns `0`, the container structurally dies immediately as designed.

1. Force-restart explicitly intercepting the broken entrypoint: `docker run -it --entrypoint /bin/sh `.
2. Interrogate the structural breakdown via `docker logs `.
3. If it requires a persistently hung process, append a long-running tail command: `CMD ["tail", "-f", "/dev/null"]`.

Q: What are Dockerfile security best practices?

- Intentionally execute a `USER non-root` directive mitigating root-level container escalation boundaries.
- Mandate trusted base environments universally specifying static SHA256 image hashes, not volatile `:latest` tags.
- Prohibit `COPY . .` injecting malicious local `.env` keys outright.

- Impose Read-Only Root Filesystems limiting write operations singularly to volatile `/tmp` partitions.

Q: What is the purpose of .dockerignore?

Fundamentally acts inversely parallel to `.gitignore`. It universally blocks massive useless directories (like `node_modules`, `.git`, or localized `build/` files) from persistently being injected explicitly into the Docker build context daemon. It structurally protects against accidentally shipping developers' `.env` plaintext credentials violently into globally public images.

Q: Why is COPY . . risky in CI/CD pipelines?

If `.dockerignore` isn't flawlessly configured, `COPY . .` indiscriminately dumps exactly everything into the container. Most catastrophically, this accidentally ships `.git/` history (expanding the image by megabytes) and highly classified developer `.env` files/AWs keys straight into the finalized public image layers, radically expanding potential cyber-attack vulnerability vectors.

Q: How do you ensure deterministic Docker builds?

1. Universally bind foundational `FROM` targets explicitly directly against irreversible SHA256 cryptographic digests, utterly ignoring arbitrarily floating `:1.0` semantic tags.
2. Persistently enforce explicit standard package locking (`npm ci` over `npm install`).
3. Remove unpredictable temporal directives indiscriminately pulling `apt-get upgrade` independently during the build process.

Q: Explain Docker architecture.

Docker uses a **client-server architecture**:

- **Docker Client** (`docker` CLI) — sends commands via REST API
- **Docker Daemon** (`dockerd`) — listens for API requests, manages images, containers, networks, and volumes
- **Docker Registry** (Docker Hub / ECR) — stores and distributes images
- **containerd** — low-level container runtime; manages container lifecycle
- **runc** — OCI runtime that actually creates/runs containers using Linux namespaces + cgroups

Flow: `docker run` → Docker Client → Daemon → containerd → runc → container process

---

Q: Write a basic Dockerfile.

FROM node:20-alpine WORKDIR /app COPY package*.json ./ RUN npm ci --only=production COPY . . EXPOSE 3000 CMD ["node", "server.js"]

- **FROM** — base image (alpine = minimal footprint)
- **WORKDIR** — sets working directory inside container
- **COPY package*.json first** — leverages layer cache; dependencies only reinstall when package.json changes
- **RUN npm ci** — reproducible install (uses lockfile, fails if out of sync)
- **EXPOSE** — documents the port (does not publish it)
- **CMD** — default command when container starts

Q: If a Docker image is built and you manually install missing packages inside a container, how will you convert it into a new Docker image?

Use `docker commit` to snapshot the container's current state into a new image:

While container is running or stopped docker commit <container_id> my-new-image:v2

With author and message

docker commit -a "yourname" -m "added curl and wget" <container_id> my-new-image:v2

Push to registry

docker tag my-new-image:v2 myrepo/my-new-image:v2 docker push myrepo/my-new-image:v2

⚠️ Best practice: Always update the Dockerfile instead. `docker commit` creates an opaque layer — not reproducible, hard to audit, and breaks CI/CD pipelines.

Q: Difference between CMD, ENTRYPOINT, and RUN in Docker.

Instruction	When it runs	Purpose	Overridable?
RUN	Build time	Executes commands to build image layers (install packages, compile code)	N/A — part of image
CMD	Container start	Default arguments/command. Easily overridden by `docker run <image> <override>`	Yes — fully replaced
ENTRYPOINT	Container start	Fixed executable — defines what the container IS. CMD becomes arguments to it.	Only with `--entrypoint` flag

Together: ENTRYPOINT + CMD ENTRYPOINT ["python", "app.py"] CMD ["--port", "8080"] # default arg, overridable

docker run myimg --port 9090 → runs: python app.py --port 9090

Q: What is kubeconfig in Kubernetes?

kubeconfig is a YAML file (default: `~/.kube/config`) that `kubectl` uses to authenticate and communicate with Kubernetes clusters.

- **clusters** — API server endpoints and CA certificates
- **users** — credentials (client cert, token, OIDC)
- **contexts** — named combination of cluster + user + namespace

Switch context (cluster) kubectl config use-context my-eks-cluster

View current context

kubectl config current-context

Merge multiple kubeconfigs

KUBECONFIG=~~/.kube/config:~~/.kube/eks-config kubectl config view --merge --flatten

In EKS: run `aws eks update-kubeconfig --name <cluster> --region <region>` to auto-populate kubeconfig.

Q: How do you handle secrets in Kubernetes?

Kubernetes **Secrets** store sensitive data (passwords, tokens, certs). Base64-encoded — *not encrypted by default*.

- **Native Secret:** `kubectl create secret generic db-pass --from-literal=password=s3cr3t`
- **Mount as env var or volume** in pod spec
- **Enable Encryption at Rest:** Configure `EncryptionConfiguration` with KMS provider in kube-apiserver
- **External Secrets Operator (ESO):** Syncs secrets from AWS Secrets Manager / Azure Key Vault into K8s Secrets automatically
- **CSI Secrets Store Driver:** Mounts secrets directly from vault as files — never stored in etcd

Best practice: Never commit secrets to Git. Use ESO + Secrets Manager for production.

Q: How do you manage environment variables in Kubernetes?

Three main approaches:

- **Hardcoded in pod spec** (dev/non-sensitive only): `env: [{name: APP_ENV, value: production}]`
- **ConfigMap** — for non-sensitive config: `envFrom: [{configMapRef: {name: app-config}}]`
- **Secret** — for sensitive values: `envFrom: [{secretRef: {name: db-secret}}]`

Use `valueFrom.fieldRef` to inject pod metadata (pod name, namespace) as env vars — useful for logging context.

Q: How do you fetch secrets from Azure Key Vault and use them in Kubernetes?

Two main approaches when using AKS (or any K8s with Azure identity):

- **Azure Key Vault Provider for Secrets Store CSI Driver:**
  1. Install the CSI driver addon on AKS
  2. Create a `SecretProviderClass` referencing Key Vault + secret names
  3. Mount as volume in pod — secrets appear as files
  4. Optionally sync to K8s Secret object via `secretObjects` in SecretProviderClass
- **External Secrets Operator (ESO):** Define `ExternalSecret` CRD → ESO polls Key Vault → creates/updates K8s Secret automatically

apiVersion: secrets-store.csi.x-k8s.io/v1 kind: SecretProviderClass metadata: name: azure-keyvault-secret spec: provider: azure parameters: keyvaultName: my-kv objects: | - objectName: db-password objectType: secret

---

Q: What is Ingress and how do you use it?

Ingress is a Kubernetes API object that manages external HTTP/HTTPS access to services inside the cluster. It provides routing rules, TLS termination, and host/path-based routing — a single entry point instead of exposing every service with a LoadBalancer.

- **Requires an Ingress Controller** (e.g., NGINX, AWS ALB Controller, Traefik) — Ingress alone does nothing
- **Path routing:** `/api → api-service`, `/web → frontend-service`
- **Host routing:** `api.example.com → api-service`, `app.example.com → frontend-service`
- **TLS:** reference a Secret containing cert+key

apiVersion: networking.k8s.io/v1 kind: Ingress metadata: name: app-ingress annotations: kubernetes.io/ingress.class: nginx spec: rules: - host: app.example.com http: paths: - path: /api pathType: Prefix backend: service: {name: api-svc, port: {number: 80}}

🏗️ Infrastructure as Code & CI/CD

Q: Have you worked with IaC?

Yes — Terraform primary, CloudFormation for AWS-native (Control Tower, Service Catalog).

Q: When CloudFormation vs Terraform?

- **CFN**: 100% AWS, Control Tower, StackSets, Service Catalog, no TF expertise
- **Terraform**: Multi-cloud, provider-agnostic, richer modules, better state mgmt, more expressive HCL

Q: Will you still choose Terraform?

Yes — unless hard CFN requirement (StackSets, etc.). Terraform's ecosystem, modularity, and provider breadth make it the stronger long-term investment.

Q: How would you architect a Terraform directory structure?

terraform/ ├── modules/ (vpc, eks, rds, iam) ├── environments/ │ ├── dev/ (main.tf, variables.tf, terraform.tfvars) │ ├── staging/ │ └── prod/ ├── global/ (shared IAM, S3) └── backend.tf (remote state)

Each env is its own root — prevents blast radius.

Q: Open source or enterprise Terraform?

Open source (OpenTofu for newer projects). Sufficient for our use cases.

Q: Do you write locally and deploy directly to prod?

No — local → feature branch → PR → CI runs `terraform plan` → review → merge triggers `terraform apply` via GitHub Actions.

Q: How to rollback infrastructure with Terraform?

- **Git revert** + re-apply
- **State snapshots**: S3 versioning → restore previous state
- **Targeted destroy** + re-apply previous config
- Best practice: small incremental PRs reduce rollback scope

Q: Ansible in real projects?

- Config mgmt (packages, users, security)
- Deployment (rolling updates)
- Post-Terraform provisioning
- Patch management
- Environment setup

Q: How did you deploy SSL?

- **ACM + ALB**: Free cert, auto-renew, ALB terminates SSL
- **Certbot**: Let's Encrypt for direct EC2
- **cert-manager**: K8s automated issuance

Best: HTTPS redirect, HSTS, disable TLS 1.0/1.1.

Q: How do you design zero-downtime deployments in AWS?

- Use managed rolling updates in EC2 Auto Scaling Groups or Kubernetes Deployments. Ensure pod/instance health checks map properly to load balancers.
- Leverage Blue-Green patterns for high-risk changes, redirecting traffic via Route 53 weights or ALB listener rules.
- Decouple databases: use schema expansions (additive changes only) before deploying new application code that depends on those changes.

Q: What is blue-green deployment in AWS?

A strategy where two identical environments (Blue and Green) run concurrently. The current version runs on Blue. New code is deployed to Green and tested safely. When ready, the load balancer or DNS is swapped to instantly direct all user traffic to Green. If issues arise, rolling back is as simple as flipping the toggle back to Blue.

Q: What is canary deployment?

Deploying a new application version natively to a small subset of live users before rolling it out globally. The load balancer (e.g., ALB weighted target groups) points 5% of traffic to the new version. Observability tools monitor error rates and latency. If thresholds remain pristine, traffic incrementally increases to 100%.

Q: Explain a Terraform folder structure you have used in projects

├── modules/ # Reusable standard components (vpc, eks, rds) ├── environments/ │ ├── dev/ # main.tf, variables.tf, terraform.tfvars │ ├── staging/ │ └── prod/ # Each env has its own isolated remote state ├── global/ # Shared resources (IAM, Route53, S3 backend buckets) └── backend.tf # State locking configuration

---

Q: How will you create 5 EC2 instances with different IAM roles and instance types using Terraform?

Use a `for_each` loop iterating over a robust map variable. Do not use `count`, as removing indices destroys subsequent resources.

variable "servers" { type = map(object({ type = string, role = string })) } resource "aws_instance" "server" { for_each = var.servers instance_type = each.value.type iam_instance_profile = each.value.role }

Q: How do you handle Terraform versioning in a project?

- Define exact `required_version` blocks inside the Terraform configuration to block incompatible binaries.
- Define strict provider versions in the `required_providers` block.
- Use `tfenv` or `asdf` locally so developers are transparently assigned the correct CLI binary on a per-project basis.
- In the CI/CD pipeline, run container images pinned to the specific version.

Q: You want to delete only an EC2 instance from Terraform — which argument/command will you use?

To surgically destroy a single resource without affecting the rest of the infrastructure, use state targeting: `terraform destroy -target=aws_instance.my_server`.

Note: Targeting circumvents Terraform's complete lifecycle understanding and is an anti-pattern for normal automation; it's reserved for exceptional troubleshooting.

Q: Which deployment strategies have you implemented? (Rolling, Blue-Green, Canary, etc.)

- **Rolling (Default):** EKS pod deployments. Best for rapid iteration where 100% backward compatibility is guaranteed.
- **Canary:** Istio/ALB routing. Used for critical tier-1 microservices where exposing large user bases to untested changes is risky.
- **Blue-Green:** Database-intensive platforms where rollbacks must be instantaneous via DNS/LB flips to avoid long draining states.

Q: What is a Terraform module and why do we use it?

A module is a logical container for multiple interconnected resources grouping them into a single highly-reusable abstraction. We use them to enforce standard organizational security practices (i.e. "Golden Paths")—ensuring that every time developers request an S3 bucket, it is automatically bundled with KMS encryption and proper block public access rules.

Q: What is the difference between data block and resource block in Kubernetes/Terraform?

- **Resource block:** Used to declare, provision, and maintain new infrastructure components exclusively driven by your Terraform state.
- **Data block:** Read-only mechanism allowing Terraform to query and absorb existing infrastructure parameters dynamically (e.g. querying the most recent Amazon Linux AMI ID, or fetching an existing VPC ID).

Q: Which CI/CD pipeline have you used (Declarative / Scripted) and why?

Declarative Pipelines (like GitHub Actions YAML or GitLab CI) over scripted variants. Declarative inherently treats the pipeline definition as code, residing directly beside application code. It mitigates pipeline configuration drift, standardizes peer reviews for CI/CD alterations, and provides robust visual integrations straight into Git repositories.

Q: You created an S3 bucket using Terraform, now you want to manage it manually from AWS Console.

Do not simply delete the module, as `terraform apply` will destroy the actual bucket. Instead, strictly sever Terraform's linkage to the resource by running:
`terraform state rm aws_s3_bucket.my_bucket_name`
This removes the structural metadata from the remote state while leaving the bucket perfectly unharmed in AWS.

Q: What is Canary Deployment and how do you implement it in Kubernetes?

A strategy where new code is deployed to a small fraction of real users (e.g. 5%) before a wider rollout. In Kubernetes, it's typically implemented using an Ingress Controller (like Istio or NGINX) enforcing traffic-split percentages across two separate Deployments, or managed natively via tools like Argo Rollouts.

Q: What are Terraform Workspaces and when should you use them?

Workspaces represent independent state files bound to the exact same configuration code directory. Calling `terraform workspace new dev` isolates a duplicate infrastructure stack laterally. However, for massive enterprise

environments requiring strict variable delineations and blast radius limits, adopting completely isolated directory structures is historically safer.

Q: How does Terraform state management work internally?

TF State is a JSON schema that maps declared logical configurations precisely to live infrastructure provider IDs. When you modify code, Terraform executes a differential read against the State file, discerns precisely what parameter drifted, and generates an exact algebraic plan to reconcile the physical environment with the code.

Q: What is terraform import and in which real-world scenario is it used?

`import` binds unmanaged, manually constructed infrastructure (e.g., an S3 bucket someone hastily produced in the console during an incident) back under systemic terraform control. It synchronizes the external ID straight into the TF state file, allowing subsequent automated management without forcing a destructive re-creation.

Q: How would you design a secure CI/CD pipeline for production?

- OIDC federation for strict short-lived cloud authentication (no static keys).
- Enforcing Branch Protection Rules explicitly necessitating mandatory multi-party SME approvals.
- Aggressive Shift-Left SAST scanning tooling positioned pre-merge.
- Deployments restricted entirely to isolated, private-VPC ephemeral runners bereft of ingress internet availability.

Q: How do you handle secret management in pipelines?

Placing API keys persistently into environment variables or repository configuration stores constitutes an instant disqualification. I implement externalized fetching architectures using HashiCorp Vault or AWS Secrets Manager. The CI runner authentically identifies itself via OIDC, extracts the strictly-permissioned token temporarily into memory, and injects it dynamically exactly at runtime execution.

Q: Secrets management in CI/CD - what disqualifies you instantly

Answering: "*I generate an AWS IAM Access Key and embed it securely into GitHub Secrets.*" This unequivocally disqualifies you. The correct architectural paradigm entirely repudiates static standing credentials. CI/CD must exclusively harness intrinsic IAM OIDC Federation trusts, generating strictly short-lived, transient bearer tokens immediately dissolving upon pipeline completion.

Q: Is a Webhook inbound or outbound, and how does it work in CI/CD systems?

A Webhook represents a dynamically triggered automated *outbound* HTTP POST transmitting payload intent instantly across domains. In CI/CD architectures, an explicit Git branch push instantly triggers GitHub servers conceptually projecting an outbound webhook directly targeting your respective inbound Jenkins/ArgoCD endpoint listener, thereby initiating cascading pipeline integrations flawlessly.

Q: How do you integrate GitHub Actions with AWS for CI/CD pipelines?

Establish an immediate AWS IAM OIDC Provider establishing distinct cryptographically secure trust explicitly with Microsoft's GitHub Identity tokens. Institute the `configure-aws-credentials` open-source action strategically calling STS within a pipeline runner, obtaining ephemeral privileges purely defined exclusively targeting dedicated build artifacts.

Q: A manual change was made to an AWS resource in production causing Terraform drift, and the next terraform apply wants to recreate the resource. How would you reconcile the infrastructure without downtime?

Do not run `apply` if it triggers a destructive recreate. If the manual change is the new desired state, update the Terraform code (e.g., adding the new tags/settings) to match reality, then run a `plan` to verify 0 changes. If the code is correct and the manual change was a mistake, run `terraform apply` to overwrite the manual drift back to the original code state. If the resource ID fundamentally changed, use `terraform import`.

Q: Multiple Terraform modules across repositories depend on each other and a deployment fails due to dependency conflicts. How would you design a reliable deployment strategy?

Tightly bound modules in separate repos are an anti-pattern. Decouple them by using robust **remote state data sources** (`terraform_remote_state` or `aws_ssm_parameter`) purely to pass IDs/ARNs between isolated layers (e.g., Network, Data, App). For orchestration, adopt a wrapper framework like **Terragrunt** to explicitly map and execute sequential dependency graphs.

Q: You need to deploy a new version of a critical microservice in production without any downtime. Which deployment strategies would you implement?

Blue-Green Deployment: Spins up the V2 environment cleanly parallel to V1, tested silently, and then traffic is instantaneously shifted over at the ALB/Route53 layer. Zero downtime, immediate rollback.
Canary Deployment: Safely redirects exactly 5% of real traffic to V2, actively monitors P95 latencies and 5xx errors, and progressively dials the weight loop to 100%.

Q: Your deployment is not updating properly (rolling update stuck). What could be wrong?

Look at `kubectl rollout status deployment `. If stuck, a new ReplicaSet was created but its pods cannot become `Ready`.
1. Failing Readiness Probes (app is booting but returning 500s).
2. `ImagePullBackOff` (spelled the image tag incorrectly, or missing ECR auth).
3. Kubernetes halted the rollout because `maxUnavailable` constraints were hit and no new pods could pass health checks to proceed.

Q: If Terraform state file has older entries and infra was created manually, how will you add it to Terraform?

Use `terraform import` to bring manually created resources under Terraform management:

General syntax terraform import <resource_type>.<name> <resource_id>

Example: import an existing EC2 instance

terraform import aws_instance.web i-0abc123def456

After import, generate/write matching HCL config

Run plan to verify no unintended changes

terraform plan

- Write the matching resource block in `.tf` before importing — Terraform needs the config to reconcile against
- **Terraform 1.5+:** Use `import` blocks in HCL for declarative, code-reviewed imports
- After import, run `terraform plan` — a clean plan (no changes) means state matches reality
- For bulk imports: use `terraformer` (open-source tool) to reverse-engineer existing infra into HCL + state

---

Q: If there are 1000+ changes to be managed in Terraform state, how will you handle it?

Managing large Terraform state requires splitting and structuring to reduce blast radius and improve performance:

- **Split into smaller state files (state isolation):** Break monolithic state into workspaces or separate root modules per domain (networking, compute, security). Each has its own isolated state.
- **Use `terraform state mv`:** Move resources between states without destroying and recreating them
- **Targeted applies:** `terraform apply -target=aws_instance.web` — only applies specific resources. Use sparingly; can cause drift.
- **Remote state with locking:** S3 + DynamoDB (AWS) prevents concurrent modifications in team environments
- **State surgery tools:** `terraform state rm` (remove from state without destroying infra), `terraform state pull/push` for manual edits

At 1000+ resources, the real fix is module decomposition — never let a single state file grow unbounded. Each team/domain should own their own state.

🗄 Databases & Storage

Q: Have you hosted a website on S3?

Yes — static React/HTML sites on S3 with CloudFront.

Q: What configurations for publicly available S3 website?

- Enable **Static Website Hosting** (index + error docs)
- Bucket Policy allowing `s3:GetObject` for `"Principal":"*"`
- Disable Block Public Access
- Use S3 website endpoint (not REST)
- Optional: CloudFront for HTTPS, caching, custom domain

Q: Anything else?

- Custom error document for SPA routing
- Route 53 Alias to CloudFront
- Versioning for rollback
- **OAC** with CloudFront — bucket stays private, CF serves publicly

---

Q: What is a parameter group in Amazon RDS?

Container for engine config values — memory, query cache, timeouts, logging. **DB Parameter Group** (individual instances) and **DB Cluster Parameter Group** (Aurora). Default is non-modifiable; create custom. Dynamic vs static (reboot) params.

Q: RDS can't connect to S3?

- IAM Role (s3:PutObject + trust for rds.amazonaws.com)
- Bucket Policy
- KMS (GenerateDataKey + Decrypt)
- S3 Gateway Endpoint if no internet
- SG/NACLs: outbound 443

Q: Configure Grafana dashboards?

- Data Sources + Panels (PromQL/SQL)
- Variables ($variable syntax)
- Dashboard as Code (JSON in Git + provisioning YAML)

📊 Observability, DevOps & Operations

Q: Anything apart from that?

EKS requires add-ons — CoreDNS, kube-proxy, VPC CNI, LB controller — each adding operational and cost overhead. ECS handles much natively.

Q: Any connectivity failures you resolved?

EKS pod unable to reach RDS. Root cause: RDS Security Group not allowing inbound from EKS node SG. DNS worked, so wasn't immediately obvious — used `telnet` / `nc` to confirm port-level block.

Q: Any long-standing issue?

Intermittent DNS failures in EKS — CoreDNS overloaded during traffic spikes. Fixed by tuning CoreDNS replicas, adding **NodeLocal DNSCache**, and setting resource requests/limits.

Q: What would you implement to optimize cloud costs?

- Cost Explorer + Budgets
- Trusted Advisor + Compute Optimizer
- Reserved Instances / Savings Plans
- Spot Instances for fault-tolerant workloads
- S3 Lifecycle Policies → Glacier
- Auto Scaling + scheduled scaling (off-hours)
- Tagging strategy via SCPs

Q: Is Trusted Advisor free?

Basic checks are free. Full set (cost optimization, fault tolerance, performance) requires **Business or Enterprise Support plan**.

Q: Will you still use Spot for batch jobs?

Yes — Spot is excellent for fault-tolerant batch. AWS Batch and Karpenter handle interruptions via checkpointing or requeuing.

Q: What if that instance type isn't available in that region?

Use **Spot Fleet** or **Karpenter with instance diversification** — specify multiple compatible types (m5.large, m5a.large, m4.large). Never rely on a single type.

Q: Steps to investigate and remediate an issue?

- **1. Triage**: CloudWatch alarms, dashboards, recent deployments
- **2. Logs**: CloudWatch Insights, pod logs
- **3. Metrics**: CPU, memory, network, error rates
- **4. Tracing**: X-Ray/Jaeger for distributed tracing
- **5. Events**: kubectl describe, CloudTrail, IAM changes
- **6. Remediate**: Rollback, scale up, fix config
- **7. Post-mortem**: Timeline, root cause, preventive actions

---

Q: Grafana dashboard types in DevOps?

- Infra (CPU/mem/disk via Node Exporter)
- K8s (pods, nodes via kube-state-metrics)
- APM (RED metrics, P95)
- CI/CD (DORA)
- Logs, DB, Cost

Q: Email alerts from Grafana?

- SMTP in grafana.ini
- Contact Point (Email)
- Alert Rule (query + condition + for duration)
- Notification Policy
- Go templates for email body

Q: Common alert conditions?

- CPU >85% / Memory >90% (5 min)
- Disk >80% warn / >90% crit
- Pod restarts (CrashLoopBackOff)
- 5xx rate spike / P95 latency
- up==0 >2 min / cert expiry / DB lag

Q: What is AWS Config and why is it used?

AWS Config acts as a sophisticated DVR for cloud configuration. It continually audits, records, and evaluates the configurations of your AWS resources, documenting how they evolve over time. It's heavily used by compliance and security teams to trigger alerts when a resource violates organizational guidelines (e.g. someone opens SSH port 22 globally).

Q: What is AWS GuardDuty?

Amazon GuardDuty is an ML-powered threat detection capability bridging the fundamental gap between network visibility and IAM insight. It continuously ingests VPC Flow Logs, CloudTrail Events, and DNS logs, producing actionable findings regarding unusual API activities, potential outbound crypto-mining connections, or heavily compromised EC2 instances.

Q: How do you set up an alert policy in GKE if any pod crashes?

Establish a Cloud Logging sink expressly filtering for systemic K8s signals like `OOMKilled` or `CrashLoopBackOff`, and translate that query directly into a custom Log-based Metric. Subsequently, formulate a Cloud Monitoring Alert Policy attached to that metric specifically triggering your organization's PagerDuty integration upon any abnormal deviation.

Q: What is AWS Systems Manager (SSM)? How can you deploy or manage applications on EC2 using SSM?

A comprehensive operational suite. It fundamentally replaces insecure bastion hosts with **SSM Session Manager** (facilitating auditable terminal interactions without open ports). Applications are securely orchestrated rapidly onto thousands of EC2 targets through **SSM Run Command** or permanently enforced via **State Manager** documents containing bootstrap automations.

Q: What is MLOps, and which tools are commonly used in MLOps pipelines?

MLOps translates robust DevOps automation philosophies structurally onto Machine Learning development lifecycles. It concerns programmatic model-training orchestration, continuous evaluation accuracy verifications, and streamlined live-endpoint model deployments. Industry solutions: Kubeflow, MLFlow, AWS SageMaker Pipelines.

Q: How to debug high CPU on EC2 like a senior engineer

Junior engineers intuitively guess hardware faults and force reboot without diagnostic evidence. **Senior Sequence:** Execute `top/htop` internally securing precisely the culprit PID. Extract thread metrics utilizing `jstack` or profilers. Trace fundamental kernel bottlenecks via `strace -p `. Interrogate local CloudWatch agent logging to correlate resource depletion exact timings against upstream concurrent application payload metrics.

Q: A vulnerable dependency was detected in a Docker image after deployment. What steps would you take to mitigate the risk and secure the CI/CD pipeline?

Immediate: Identify the blast radius, patch the library in the source `Dockerfile`, re-build, and trigger a rolling deployment.
Pipeline Fix: Implement proactive container vulnerability scanning (Trivy, Snyk, Inspector) directly into the CI pipeline as a rigid blocking step *prior* to pushing to the ECR registry. Continuously scan ECR repositories for newly discovered CVEs.

Q: Users report intermittent failures across microservices, but logs from individual services look normal. How would you identify the root cause?

When isolated service logs look normal, the problem is network transit, latency, or cascading timeouts. Implement **Distributed Tracing** (AWS X-Ray, Jaeger, or OpenTelemetry). Inject correlation Trace IDs into standard headers at the API Gateway, and track the exact microscopic latency of the request as it traverses across every internal microservice boundary to locate the invisible choke point.

🐧 Linux, Scripting & Git

Q: What is Node Exporter?

Prometheus exporter for HW/OS metrics on port 9100. Key: cpu, memory, disk, filesystem, network. K8s: DaemonSet. Dashboard ID 1860.

Q: What is PM2?

Production Node.js process manager — keeps alive, auto-restart, cluster mode (`-i max`), ecosystem.config.js, zero-downtime reload, `pm2 startup` + `pm2 save` .

Q: Kill a PM2 process?

pm2 stop app / pm2 delete app pm2 stop all / pm2 delete all pm2 kill # daemon + all

---

Q: Check running nodes?

pm2 list | pm2 show app | pm2 monit

Q: Kill a process in Linux?

kill PID / kill -9 PID / killall name / pkill -f "pattern"

SIGTERM first. -9 last resort.

Q: chmod 555?

5=r+x → owner=r-x, group=r-x, others=r-x. Everyone reads/executes, nobody writes.

Q: Check open ports?

ss -tlnp / netstat -tlnp / lsof -i -P -n / nmap localhost

Q: Cron job?

min hour dom month dow command 0 2 * * * /backup.sh # daily 2AM */5 * * * * /health.sh # every 5 min crontab -e / crontab -l

---

Q: Clear cache?

sync && echo 3 > /proc/sys/vm/drop_caches

App: `apt clean` , `npm cache clean --force` . DNS: `systemd-resolve --flush-caches` .

Q: Resolve merge conflicts?

- `git status` → "both modified"
- Edit markers (<<</===/>>>), keep desired
- `git add` → `git commit`
- Tools: git mergetool, VS Code conflict UI

Q: Switch to a new branch?

git checkout branch / git switch branch # existing git checkout -b new / git switch -c new # create+switch git branch -a # list all

---

Q: Which branching strategy have you used in your project and why?

Trunk-Based Development with robust feature flags. It mandates that developers frequently merge extremely small, testable chunks sequentially into the `main` trunk branch multiple times per day. It directly avoids the catastrophic "Merge Hell" commonly seen in complex GitFlow releases and is the absolute prerequisite philosophy for functioning Continuous Deployment.

Q: Explain the booting process of Linux.

Linux boot sequence — 6 stages:

1. **BIOS/UEFI** — Power-on self-test (POST), locates bootable device
2. **Bootloader (GRUB2)** — Loads the kernel into memory from `/boot`, passes kernel parameters
3. **Kernel initialization** — Kernel decompresses itself, initializes hardware drivers, mounts root filesystem (read-only first)
4. **initramfs / initrd** — Temporary root filesystem with tools needed to mount the real root FS (handles LVM, LUKS, etc.)
5. **init / systemd (PID 1)** — First process started by kernel. Systemd reads unit files, starts services in parallel based on dependencies
6. **Runlevel / Target** — Systemd reaches the target (e.g., `multi-user.target` or `graphical.target`), spawns login prompt

Key command: `systemd-analyze blame` — shows which services slowed down boot.

Q: What are services in Linux?

A **service** (daemon) is a background process managed by the init system (systemd). It runs without user interaction.

Start / stop / restart systemctl start nginx systemctl stop nginx systemctl restart nginx

Enable at boot / disable

systemctl enable nginx systemctl disable nginx

Check status

systemctl status nginx

List all services

systemctl list-units --type=service

Services are defined as **unit files** in `/etc/systemd/system/` or `/lib/systemd/system/`. They specify ExecStart, dependencies, restart policies, and the user they run as.

Q: How do you check open ports in Linux?

ss — modern replacement for netstat ss -tlnp # TCP listening ports with process name ss -ulnp # UDP listening ports

netstat (legacy, may need net-tools)

netstat -tlnp

lsof — list open files/sockets

lsof -i :80 # what's using port 80 lsof -i TCP -P -n

nmap — scan from outside

nmap -sT localhost

- `-t` TCP, `-u` UDP, `-l` listening, `-n` numeric, `-p` show process
- Best for quick check: `ss -tlnp | grep LISTEN`

Q: Explain the Linux file system structure.

Directory	Purpose
`/`	Root — top of the entire hierarchy
`/bin`	Essential user binaries (ls, cp, bash)
`/sbin`	System binaries (fsck, ifconfig) — for root
`/etc`	System-wide configuration files
`/home`	User home directories
`/var`	Variable data: logs, spool, tmp persisted across reboots
`/tmp`	Temporary files, cleared on reboot
`/proc`	Virtual FS — kernel/process info (e.g., `/proc/cpuinfo`)
`/sys`	Virtual FS — hardware/driver info exposed by kernel
`/usr`	User programs, libraries, docs (bulk of installed software)
`/opt`	Optional/third-party software (e.g., custom installs)
`/boot`	Bootloader, kernel image, initramfs
`/dev`	Device files (block/char devices)
`/mnt`, `/media`	Mount points for external/removable storage

Q: Command to check OS version in Linux?

Most reliable — works on all distros cat /etc/os-release

Short version

lsb_release -a

Kernel version

uname -r

Full system info

uname -a

RHEL/CentOS specific

cat /etc/redhat-release

Debian/Ubuntu specific

cat /etc/debian_version

For scripting, `/etc/os-release` is the standard — it's a key=value file that all modern distros provide.

🔥 Real-World Scenarios

Q: Your Jenkins pipeline build is successful, but deployment is failing. What will you check?

Follow the deployment path from artifact to target environment

- **Artifact & Tagging:** Verify the image was actually pushed to the registry (ECR, Nexus, Artifactory) and the deployment manifest is referencing the correct tag.
- **Auth & Secrets:** Check if the deployment agent has the right IAM role, kubeconfig, or SSH keys. Secrets might be rotated but not updated in CI.
- **Network Connectivity:** Can the Jenkins agent talk to the target environment? Did a security group or NACL change?
- **Deployment Environment Health:** Check disk space on target nodes or check if the Kubernetes cluster API is reachable and responding.
- **Logs:** Read the exact deployment stage output. Identify if it's an authorization error (401/403) or a connection timeout (504).

Q: A Kubernetes pod is in CrashLoopBackOff. How will you debug it?

Systematic descending isolation: Events → Logs → State

- **Events:** `kubectl describe pod <pod-name>` to check for OOMKilled, Liveness probe failures, or image pull errors.
- **Previous Logs:** `kubectl logs <pod-name> --previous` to see why the *last* container crashed before it restarted.
- **Environment/Config:** Check if required ConfigMaps or Secrets are mounted properly and have the correct keys.
- **Dependencies:** Is the database reachable? Many apps crash immediately if they can't connect to their required services on startup.
- **Interactive Debugging:** If logs are empty, overwrite the command to `sleep 3600`, exec into the pod, and run the startup script manually to see the failure.

Q: Terraform infrastructure has drifted due to manual changes. What's your approach?

Identify, Evaluate, and Reconcile

- **Identify the Drift:** Run `terraform plan` to clearly see what was changed manually (additions, modifications, deletions).
- **Find the Cause:** Check CloudTrail or audit logs to determine *who* made the change and *why*. Was it a midnight incident response break-glass action?
- **Evaluate Path Forward:** If the manual change was a mistake, run `terraform apply` to revert the environment to the declared code state.
- **Importing Changes:** If the manual change was a necessary hotfix, update the Terraform code to match the new state, so the next `plan` shows no changes.
- **Prevention:** Implement strict IAM roles where humans only have ReadOnly access, and only the CI/CD pipeline role has Admin access to apply changes.

Q: EC2 CPU suddenly spikes in AWS. What steps will you take?

Correlate application metrics with system behavior

- **System Level:** SSH or use Systems Manager Session Manager, run `top` or `htop` to identify the specific process consuming CPU.
- **Application Level:** If it's a web server, check access logs for a traffic spike, DDoS attempt, or long-running DB queries locking up workers.
- **Instance Level:** Check CloudWatch metrics for exhausted CPU credits (if T-series burstable instances) or disk I/O bottlenecks causing high CPU wait states (iowait).
- **Immediate Action:** If part of an Auto Scaling Group, ensure scaling policies are triggering to add more instances to handle the load.
- **Remediation:** Profile the application, implement rate limiting/WAF, or upgrade the instance type if the baseline has permanently shifted.

Q: Docker container works locally but fails in production. Why?

"It works on my machine" is an environment difference problem

- **Environment Variables:** Missing, malformed, or incorrect production secrets (DB strings, API keys) that were hardcoded locally or in a `.env` file.
- **Architecture Mismatch:** Developing on an M1/M2 Mac (ARM64) and deploying to an EC2 instance running x86_64 without building multi-architecture images.
- **Network/Firewall:** The app connects to external APIs that are blocked by production VPC NAT Gateways or Security Groups.
- **Dependencies:** Volume mounts used locally aren't present in production, or differing OS packages inside a loosely defined Dockerfile.
- **Compute Limits:** The container hits memory limits (OOMKilled) in production Kubernetes because local Docker Desktop had more resources assigned.

Q: How do you ensure zero downtime deployments?

Decouple deployment from release

- **Blue/Green Deployment:** Spin up a complete parallel environment (Green) with the new version. Test thoroughly, then flip the ALB/Route 53 traffic weight to 100% Green. Rollback is instant.
- **Canary Deployments:** Gradually shift 5%, then 10%, then 100% of traffic to the new version while monitoring error rates via CloudWatch or Prometheus.
- **Rolling Updates:** In Kubernetes, update pods one by one. Ensure readiness probes are properly configured so traffic isn't routed to a pod until it's actually ready.

- **Database Strategy:** Ensure database migrations are strictly backwards-compatible (expand and contract pattern) so both old and new app versions can run simultaneously.

Q: How do you securely manage secrets in production?

Zero trust, zero static credentials, and encryption at rest

- **Secret Stores:** Use AWS Secrets Manager, Parameter Store, or HashiCorp Vault. *Never* store secrets in plaintext in Git or Kubernetes manifests.
- **Dynamic/Short-Lived Secrets:** Prefer generating ephemeral credentials (e.g. Vault AWS/DB engines) over long-lived static passwords.
- **Access Control:** Attach IAM roles directly to application instances/pods (IRSA in EKS) to fetch secrets natively, avoiding the need to pass identity tokens manually.
- **CI/CD Pipelines:** Use OIDC (OpenID Connect) for GitHub Actions to assume AWS roles instead of storing permanent `AWS_ACCESS_KEY_ID` s in the CI variables.
- **Encryption:** Ensure secrets are encrypted at rest using KMS Customer Managed Keys.